

**SIGN-BRIDGE: A REAL-TIME CNN FRAMEWORK FOR
MULTI-MODAL SIGN ALPHABET RECOGNITION**

A PROJECT REPORT

SUBMITTED BY

SABIN SASIDHARAN

(OTAYMCS012)

FOR THE AWARD OF THE DEGREE OF

MASTER OF SCIENCE (M.Sc.)

IN COMPUTER SCIENCE

(UNIVERSITY OF CALICUT)



DEPARTMENT OF COMPUTER SCIENCE

N.S.S COLLEGE OTTAPALAM

(AFFILIATED TO THE UNIVERSITY OF CALICUT)

PALAPPURAM, KERALA – 679103

JULY 2026

ACKNOWLEDGEMENT

First and foremost, I thank the Almighty God for providing me with the strength, health, and endurance to successfully complete this postgraduate project work.

I express my deepest sense of gratitude to **Neetha P C** Associate Professor and Head of the Department of Computer Science, for providing administrative support, expert guidance, and the necessary infrastructural assets throughout this development cycle.

I am profoundly grateful to my project guide, **Priyesh K. G**, Associate Professor, Department of Computer Science, for invaluable supervision, persistent encouragement, and critical evaluation of this multi-modal system architecture.

Finally, I extend my heartfelt thanks to my parents, friends, and the faculty members of the Department of Computer Science at N.S.S. College Ottapalam for their constant motivation and support.

Date:

Name: Sabin Sasidharan

DECLARATION

I hereby declare that this submission is my own authentic project work carried out under the supervision of **Priyesh K. G**, Associate Professor, Department of Computer Science, and that, to the best of my knowledge and belief, it contains no material previously published or written by another individual, nor material which has been accepted for the award of any other degree or diploma of the University of Calicut or any other institute of higher learning, except where due acknowledgement has been explicitly executed within the body text.

Place:

Date:

Signature:

Name: SABIN SASIDHARAN

Register No: OTAYMCS012

CERTIFICATE

This is to certify that the project report entitled " **SIGN-BRIDGE: A REAL-TIME CNN FRAMEWORK FOR MULTI-MODAL SIGN ALPHABET RECOGNITION**" submitted by **SABIN SASIDHARAN (Register Number: OTAYMSC012)** to the University of Calicut for the award of the degree of Master of Science (M.Sc.) in Computer Science is a bona-fide record of the project work carried out by him under my explicit supervision, training, and guidance. The contents of this report, in full or in parts, have not been submitted to any other Institute or University for the award of any other degree or diploma.

Priyesh K G

Faculty Guide

Associate Professor

Neetha P C

Head of Department

PROJECT EVALUATION REPORT OF THE EXAMINERS

Certified that the candidate was examined by us in the Project Viva-Voice Examination held on and his/her Register Number is

Examiners:

1.
2.

CONTENTS

SL NO	PAGE NO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
LIST OF ABBREVIATIONS	iv
1. INTRODUCTION	1
1.1 MOTIVATION	2
1.2 OBJECTIVE	3
1.3 SCOPE OF THE PROJECT	4
2. LITERATURE REVIEW & REQUIREMENT ANALYSIS	6
2.1 LITERATURE REVIEW	6
2.1.1 GAP IDENTIFICATION	8
2.2 REQUIREMENT ANALYSIS	8
2.2.1 EXISTING SYSTEM ANALYSIS	8
2.2.2 PROPOSED SYSTEM	9
2.2.3 FEASIBILITY ANALYSIS	10
2.2.4 FUNCTIONAL REQUIREMENTS	11
2.2.5 SYSTEM REQUIREMENTS	12
2.2.5.1 HARDWARE REQUIREMENT	12
2.2.5.2 SOFTWARE REQUIREMENT	13
3. SYSTEM DESIGN & METHODOLOGY	14
3.1 SYSTEM ARCHITECTURE	14
3.1.1 HIGH-LEVEL ARCHITECTURE DIAGRAM	14

3.1.2 MODULE DESCRIPTION	15
3.2 DATA FLOW DIAGRAM	17
3.3 UML DIAGRAMS	19
3.3.1 USE CASE DIAGRAM	20
3.3.2 SEQUENCE DIAGRAM	20
3.4 METHODOLOGY	21
3.5 MATHEMATICAL FOUNDATION	24
3.6 SUMMARY	26
4. IMPLEMENTATION	27
4.1 TOOLS AND SCRIPTS FOR IMPLEMENTATION	27
4.2 MODULE HIERARCHY AND IMPLEMENTATION STRATEGY	30
4.3 CODING	31
4.4 PROBLEMS ENCOUNTERED	34
4.5 TESTING	35
4.5.1 UNIT TESTING	36
4.5.2 INTEGRATION TESTING	37
4.5.3 PERFORMANCE TESTING	37
5. RESULTS & EVALUATION	39
5.1 ASL MODEL PERFORMANCE	39
5.2 ISL MODEL PERFORMANCE	41
5.3 SYSTEM PERFORMANCE	43
5.4 SUMMARY	44
6. CONCLUSION & FUTURE WORK	45

6.1 CONCLUSION	45
6.2 LIMITATIONS	45
6.3 FUTURE WORK	46
7. BIBLIOGRAPHY	48
8. APPENDIX	49
8.1 INPUT SCREEN DESIGN	49
8.2 OUTPUT SCREEN DESIGN	50
8.3 SAMPLE CODE	51

ABSTRACT

Sign language serves as the primary communication medium for over 430 million deaf and hard-of-hearing individuals worldwide, yet existing recognition systems consistently fail to balance accuracy with practical deployable—hardware solutions impose prohibitive costs, image-based deep learning demands GPU acceleration, and landmark-based approaches remain restricted to single sign languages without integrated text composition features.

SIGN-BRIDGE addresses this gap through a real-time, multi-modal CNN framework for recognizing both American Sign Language (ASL) and Indian Sign Language (ISL) alphabets. The system processes 21 three-dimensional hand landmarks extracted by MediaPipe, normalizing them relative to wrist position to produce compact 63-dimensional feature vectors per hand. A lightweight CNN architecture with approximately 210,000 parameters classifies these features into 26 alphabet letters, achieving inference under 5 milliseconds on standard CPU hardware—a 15,000-fold reduction in input dimensionality compared to raw image processing.

The ASL model achieves 99.92% test accuracy trained on 600 images per letter, while the ISL model achieves 100% test accuracy trained on 1,200 images per letter, both operating at 25-30 frames per second without GPU acceleration. Beyond classification, SIGN-BRIDGE provides a complete communication interface with multi-layered smart prediction filtering, geometric open palm detection for SPACE insertion, hand-change detection for natural letter repetition, browser-based voice synthesis, and keyboard shortcuts for text editing. The system deploys as a Flask-based web application operating entirely offline on any standard laptop with a webcam, requiring no GPU, internet, or specialized hardware. SIGN-BRIDGE demonstrates that state-of-the-art accuracy and practical deployable are complementary goals achievable through focused architectural decisions prioritizing geometric feature representation over raw pixel processing.

Keywords: Sign Language Recognition, CNN, MediaPipe, Hand Landmarks, Real-Time Fingerspelling, ASL, ISL, Deep Learning, Assistive Technology

LIST OF FIGURES

3.1 SIGN-BRIDGE High-Level Architecture	14
3.2 Hand Landmark Detection	16
3.3 DFD Level 0	17
3.4 DFD Level 1	18
3.5 DFD Level 1.1	18
3.6 DFD Level 1.2	19
3.7 Use Case Diagram	20
3.8 Sequence Diagram	20
3.9 CNN Model Architecture	22
4.1 Module Dependency Structure	30
5.1 ASL Training History	39
5.2 ASL Confusion Matrix	40
5.3 ASL F1 Score Per Class	41
5.4 ISL Training History	41
5.5 ISL Confusion Matrix	42
5.6 ISL F1 Score Per Class	43
8.1 Home page	49
8.2 Model Selection Page	49
8.3 ASL Recognition	50
8.4 ISL Recognition	50

LIST OF TABLES

2.1 Comparative Summary of Literature	7
2.2 Functional Requirements Specification	11
2.3 Hardware Requirements	12
2.4 Software Requirements	13
4.1 Unit Test cases	36
4.2 Integration Testing	37
4.3 Performance Testing	38

LIST OF ABBREVIATIONS

ASL	American Sign Language
ISL	Indian Sign Language
CNN	Convolutional Neural Network
ReLU	Rectified Linear Unit
API	Application Programming Interface
CLAHE	Contrast Limited Adaptive Histogram Equalization
FPS	Frames Per Second
JSON	JavaScript Object Notation
MJPEG	Motion JPEG
OpenCV	Open-Source Computer Vision Library
GPU	Graphics Processing Unit
MLP	Multi-Layer Perceptron
REST	Representational State Transfer

CHAPTER 1

INTRODUCTION

In a world increasingly connected by technology, communication remains the cornerstone of human interaction. Yet, for over 430 million individuals worldwide living with disabling hearing loss, this fundamental human need remains frustratingly out of reach. Sign language serves as the natural mother tongue for the deaf and hard-of-hearing community, with American Sign Language (ASL) being the third most studied language in United States universities and Indian Sign Language (ISL) serving millions across the Indian subcontinent. Despite this prevalence, less than one percent of the global hearing population possesses even basic sign language proficiency, creating a profound and persistent communication chasm that technology has promised to bridge for decades, yet has consistently failed to do so in any practical, accessible way.

Traditional approaches to bridging this gap have fallen short in practice. Hardware-based solutions utilizing instrumented gloves and motion sensors impose prohibitive costs ranging from \$5,000 to \$15,000 and demand cumbersome equipment unsuitable for daily social interaction. Classical computer vision techniques employing Support Vector Machines and Random Forest classifiers on handcrafted features struggle with environmental variability, losing 20-30% accuracy under changing lighting conditions or varied backgrounds. More recent deep learning approaches leveraging Convolutional Neural Networks on raw image pixels have pushed accuracy beyond 95%, but at a steep price: GPU dependency, 10,000+ training images per class, 50-200MB model sizes, and inference times exceeding 15 milliseconds per frame, rendering them impractical for real-time deployment on consumer hardware. The critical insight overlooked by existing systems is elegantly simple: hand geometry alone encodes sufficient information for alphabet-level recognition.

SIGN-BRIDGE operationalizes this insight through a landmark-based approach, extracting merely 63 three-dimensional coordinates per hand using MediaPipe's efficient on-device detection framework, enabling a lightweight CNN architecture with only 210,000 parameters to achieve inference speeds under five milliseconds on standard CPU hardware. The system distinguishes itself through multi-modal architectural design, supporting both

ASL (single-hand, 63 features) and ISL (dual-hand, 126 features) within a unified framework, achieving exceptional accuracy rates of 99.92% and 100% respectively. Beyond mere classification, SIGN-BRIDGE provides a complete text composition ecosystem: an integrated open palm gesture command for spacing, smart prediction filtering requiring 90% confidence and 75% frame consensus to eliminate false triggers, intelligent hand-change detection for natural letter repetition, voice synthesis for auditory output, and a web-based interface accessible through any standard browser without requiring GPU acceleration, specialized hardware, or internet connectivity.

1.1 MOTIVATION

The World Health Organization estimates that over 430 million people worldwide suffer from disabling hearing loss, yet less than one percent of the global hearing population possesses even basic sign language proficiency, creating a profound communication chasm. In India alone, approximately 63 million individuals experience significant auditory impairment, served by fewer than 300 certified sign language interpreters nationwide. Existing solutions have failed to bridge this gap: hardware-based approaches utilizing instrumented gloves cost between \$5,000 and \$15,000, rendering them economically inaccessible, while image-based deep learning systems demand GPU acceleration, controlled lighting, and thousands of training images per gesture—requirements that collapse in real-world environments where accuracy drops by 30-40% under varying conditions. The fundamental disconnect is that researchers have optimized for publication metrics rather than practical deployable, producing systems that function in laboratories but fail in living rooms.

Beyond classification accuracy lies a more critical gap: the distinction between recognizing a letter and enabling functional communication. Identifying a gesture as "A" solves only the technical problem; composing the word "APPLE" requires integrated spacing mechanisms, natural repetition of letters, false trigger filtering during hand transitions, and output modalities such as voice synthesis—features virtually absent from existing systems. Furthermore, the deaf community is not monolithic; American Sign Language and Indian Sign Language serve distinct populations with unique manual alphabets, yet no existing system supports both within a unified framework. Real-time performance is non-negotiable for conversational use, demanding sub-5 millisecond inference per frame to match natural

fingerspelling speeds of 3-5 letters per second. SIGN-BRIDGE addresses these interconnected needs through a lightweight landmark-based CNN architecture that processes 63 spatial coordinates per hand instead of raw pixels, achieving 99.92% ASL and 100% ISL accuracy on standard CPU hardware with integrated gesture commands, smart prediction filtering, and a browser-based interface requiring no GPU, no internet, and no specialized equipment.

1.2 OBJECTIVE

The primary objectives of SIGN-BRIDGE are as follows:

1. To develop a real-time sign alphabet recognition system using a lightweight Convolutional Neural Network architecture that processes MediaPipe-extracted hand landmarks (63 spatial coordinates per hand) rather than raw image pixels, enabling inference speeds under 5 milliseconds per frame on standard CPU hardware without GPU dependency.
2. To achieve classification accuracy exceeding 95% for both American Sign Language (ASL) and Indian Sign Language (ISL) alphabets through optimized CNN architectures, data augmentation strategies, and rigorous hyperparameter tuning, targeting performance levels suitable for practical, frustration-free daily use.
3. To implement a multi-modal recognition framework that supports both single-hand ASL fingerspelling and dual-hand ISL fingerspelling within a unified system architecture, enabling seamless mode switching without code modification, model retraining, or application restart.
4. To design an intelligent prediction control system incorporating confidence thresholding (90% minimum), frame consensus requirements (75% agreement), cooldown mechanisms between outputs, and hand-change detection for natural letter repetition, eliminating the false triggers and flickering predictions that plague traditional recognition systems.
5. To integrate gesture-based text composition through geometric rule-based detection of open palm configurations for SPACE insertion, enabling users to form complete words and sentences without requiring keyboard interaction during signing.

6. To provide an accessible, zero-barrier deployment solution through a web-based Flask interface that operates entirely offline on any standard laptop with a built-in webcam, requiring no specialized hardware, no GPU acceleration, no internet connectivity, and no technical expertise beyond basic Python installation.
7. To incorporate assistive output modalities including voice synthesis for auditory feedback of composed text and comprehensive keyboard shortcuts for text manipulation, creating a complete communication bridge between sign language input and spoken or written output.

1.3 SCOPE OF THE PROJECT

The SIGN-BRIDGE system is designed for real-time recognition of static sign language alphabets, specifically targeting fingerspelling applications where users produce individual letters sequentially to spell out words. The scope encompasses recognition of all 26 letters (A-Z) of both American Sign Language and Indian Sign Language manual alphabets using static hand postures captured through a standard webcam. The system employs MediaPipe for extracting 21 three-dimensional hand landmarks per detected hand, processes these 63 spatial coordinates per hand through a custom lightweight Convolutional Neural Network, and delivers predictions within 5 milliseconds per frame on standard CPU hardware. The complete pipeline includes landmark extraction, wrist-relative normalization, CNN classification, smart prediction filtering with confidence thresholding, and gesture-based SPACE insertion through geometric open palm detection. The web-based interface operates entirely on localhost without internet dependency, accumulates recognized letters into words, provides voice synthesis for composed text, and supports keyboard shortcuts for text manipulation.

The system does not extend to dynamic gesture recognition involving motion trajectories, continuous signing streams without deliberate pauses between letters, or word-level signs that represent complete concepts. It excludes non-manual signals such as facial expressions and body posture that convey grammatical information in natural sign language, and does not support sign languages beyond ASL and ISL. Personalization for individual signing styles, mobile platform deployment, and operation on embedded devices fall outside the current scope. Recognition performance depends on adequate ambient lighting, a single

user within the camera frame, and hands that are not significantly occluded by clothing or overlapping gestures. The ISL mode requires both hands visible for optimal accuracy, while single-hand ISL signs are accommodated through zero-padding. The system processes approximately 3-5 letters per second under optimal conditions, suitable for deliberate fingerspelling though not matching natural conversational signing speed.

CHAPTER 2

LITERATURE REVIEW & REQUIREMENT ANALYSIS

2.1 LITERATURE REVIEW

Sign language recognition research has evolved through four distinct generations. Zhang et al. (2011) pioneered sensor-based recognition using electromyography and accelerometer sensors, achieving 85.3% accuracy but requiring \$2,000 hardware with periodic recalibration. Kumar et al. (2017) transitioned to depth sensing via Microsoft Kinect with SVM classification reaching 90.2% on ISL gestures, though infrared-based sensing failed under sunlight and required fixed 1.5-meter mounting distance. These hardware-dependent approaches demonstrated accurate recognition is achievable but fundamentally misaligned with user needs for invisible, mobile solutions.

Classical computer vision eliminated hardware using camera-based processing. Pugeault & Bowden (2011) combined skin segmentation with HOG features and Random Forest achieving 75% accuracy, identifying lighting invariance as the primary barrier. Barczak & Reyes (2012) applied SVM-RBF on Fourier descriptors reaching 82.4% but failed on internally-configured letters like E/S due to contour features discarding internal finger structure. Ding & Martinez (2018) achieved 88.7% using PCA on silhouettes but required uniform black backgrounds impossible outside laboratories.

Deep learning on raw images eliminated manual feature engineering. Ameen & Vadera (2019) trained a 15-layer CNN achieving 90% accuracy but required GPU inference at 12ms per frame, degrading to 72% under varied lighting. Bantupalli & Xie (2018) leveraged InceptionV3 transfer learning reaching 93.2%, though the 23M-parameter model occupied 92MB. Wadhawan & Kumar (2020) combined CNN-LSTM for dynamic ISL achieving 95.7%, requiring 8GB GPU memory at only 8 FPS. Srivastava et al. (2022) used MobileNetV2 for mobile deployment achieving 91.5% but dropping to 78% under low light, confirming architectural efficiency cannot compensate for pixel-based photometric sensitivity.

Landmark-based approaches address deployment barriers through sparse geometric representation. Halder & Tayade (2021) extracted 42 MediaPipe features with Random

Forest achieving 88% accuracy at sub-2ms inference, though omitting depth coordinates reduced discrimination. Mittal et al. (2022) incorporated full 3D coordinates through MLP reaching 94.3% on ISL digits at 1.8ms, limited to digit-only recognition. Gupta & Kumar (2023) applied graph convolutional networks on landmark sequences achieving 96.1% on dynamic words, but requiring 30+ temporal frames introduced latency unsuitable for fingerspelling.

Recent advances (2024-2025) continue narrowing the accuracy-deployable gap. Patel & Shah (2024) applied transformers to MediaPipe landmarks achieving 96.8% but with 3x higher computational cost than CNNs. Rahman et al. (2024) combined YOLOv8 detection with CNN classification reaching 94.5%, though the two-stage pipeline doubled inference latency. Chen & Wang (2024) developed lightweight landmark-based CNN achieving 95.3% at 2.1ms—closest to SIGN-BRIDGE's approach but limited to single-language ASL. Kaur & Singh (2025) explored RGB-Depth fusion reaching 97.2%, requiring specialized depth cameras absent from consumer devices. Lee et al. (2025) demonstrated Raspberry Pi deployment at 93.8%, proving offline feasibility but with 6% lower accuracy than SIGN-BRIDGE.

AUTHORS	YEAR	METHOD	ACCURACY	REAL-TIME	MULTI-LANG	GPU
Zhang et al.	2011	EMG Sensors	85.3%	yes	No	No
Pugeault & Bowden	2011	HOG + RF	75.0%	no	No	No
Bantupalli & Xie	2018	InceptionV3	90.0%	no	No	Yes
Ameen & Vadera	2019	CNN Images	93.2%	partial	No	Yes
Halder & Tayade	2021	MediaPipe + RF	88.0%	yes	No	No
Mittal et al.	2022	MediaPipe + MLP	94.3%	yes	No	No
Chen & Wang	2024	Lightweight CNN	95.3%	yes	No	No
Lee et al.	2025	Edge AI	93.8%	yes	No	No

Table 2.1 Comparative Summary of Literature

2.1.1 GAP IDENTIFICATION

- G1: No system combines MediaPipe landmarks with a dedicated CNN architecture for full 26-letter alphabet recognition—prior work uses either Random Forest/MLP on landmarks or CNN on raw images
- G2: Every reviewed system targets a single sign language exclusively; no unified framework supports both ASL and ISL simultaneously through configurable architecture
- G3: All prior systems treat classification as terminal output without integrated gesture-based spacing, prediction stability filtering, or voice synthesis for text composition
- G4: Existing systems output raw per-frame predictions causing flickering and false triggers; no confidence thresholding, frame consensus, or cooldown mechanisms exist
- G5: Image-based and transformer systems require GPU or specialized hardware; landmark-based systems lack accessible offline web deployment.

2.2 REQUIREMENT ANALYSIS

The literature review identifies persistent gaps that directly inform the requirements SIGN-BRIDGE must satisfy. This section translates research findings into actionable system specifications.

2.2.1 EXISTING SYSTEM ANALYSIS

Despite four generations of technological evolution, sign language recognition systems consistently fail to transition from laboratory prototypes to practical communication tools. Sensor-based systems demand \$2,000-\$15,000 wearable hardware and controlled environments. Classical computer vision approaches degrade 20-30% under real-world lighting. Deep learning image-based systems require GPU acceleration with 50-200MB models and exhibit 15-25% accuracy loss when lighting conditions deviate from training data. Even recent landmark-based approaches (2024-2025) remain restricted to single sign

languages without integrated text composition features. These persistent limitations directly inform the functional requirements that SIGN-BRIDGE must satisfy.

Common Limitations Across All Categories:

- Single sign language support only—no system accommodates both ASL and ISL
- Classification treated as terminal output without text composition mechanisms
- Raw per-frame predictions produce flickering results and false triggers during hand transitions
- Deployment demands GPU, controlled lighting, internet connectivity, or specialized hardware
- No confidence filtering or temporal consensus mechanisms exist for prediction stability

2.2.2 PROPOSED SYSTEM

SIGN-BRIDGE is a multi-modal landmark-based CNN framework addressing all five identified gaps through a unified architecture. The system extracts 21 three-dimensional hand landmarks via MediaPipe, generating 63 normalized spatial coordinates per hand. This 15,000-fold reduction in input dimensionality compared to raw image processing enables a lightweight 210K-parameter CNN to classify 26 alphabet letters in under 5 milliseconds on standard CPU hardware.

Core Design Decisions:

- Landmark-based processing eliminates photometric sensitivity by decoupling hand geometry from lighting conditions, skin tone, and background complexity—the network learns spatial relationships between finger joints rather than pixel intensity patterns
- Dual-language architecture accommodates ASL (single-hand, 63 features) and ISL (dual-hand, 126 features with zero-padding) through configurable input dimensionality without separate model training

- Multi-layered prediction filtering applies 90% confidence thresholding, 75% frame consensus across 15 consecutive frames, and 8-frame output cooldown to eliminate false triggers common in existing systems
- Geometric gesture detection identifies open palm configuration through five-finger extension, vertical orientation verification, and fingertip position analysis for SPACE insertion without keyboard interaction
- Browser-based deployment via Flask enables offline operation on any standard laptop with webcam—no GPU, internet, or specialized hardware required.

2.2.3 FEASIBILITY ANALYSIS

Technical Feasibility: All core components are production-tested open-source technologies. MediaPipe Hands provides robust 21-landmark detection at 30 FPS on CPU. TensorFlow 2.12 offers stable CNN inference with Keras API. OpenCV handles efficient video capture and frame preprocessing. Flask provides lightweight web serving without enterprise infrastructure. The CNN architecture contains only 210K parameters and has been empirically verified to achieve 25-30 FPS inference on consumer laptop hardware with sub-5ms per-frame latency.

Operational Feasibility: The web-based interface requires no user training beyond basic browser interaction. The system operates identically across Windows, Linux, and macOS platforms. Mode switching between ASL and ISL requires a single interface selection without code modification. Complete offline operation eliminates dependency on external services, network infrastructure, or cloud APIs. Browser-based speech synthesis provides voice output without internet connectivity.

Economic Feasibility: Total system cost is limited to a standard laptop with built-in webcam—equipment already available to the target user demographic. The entirely open-source technology stack incurs zero licensing fees or recurring subscription costs. No GPU hardware, cloud services, or specialized peripherals are required, making the system economically sustainable for long-term individual deployment.

2.2.4 FUNCTIONAL REQUIREMENTS

Functional requirements define the specific behaviours, capabilities, and operations that SIGN-BRIDGE must perform to address the gaps identified in the literature review. Each requirement is derived directly from one or more research gaps, ensuring that the system design is driven by documented deficiencies in existing solutions rather than arbitrary feature selection.

ID	Requirement	Priority
FR1	System shall detect 21 hand landmarks per frame using MediaPipe at minimum 25 FPS	High
FR2	System shall extract 63 normalized spatial coordinates per hand relative to wrist position	High
FR3	System shall classify 26 ASL letters from single-hand feature vectors through CNN inference	High
FR4	System shall classify 26 ISL letters from dual-hand feature vectors with zero-padding	High
FR5	System shall detect open palm and insert SPACE character without keyboard	High
FR6	System shall apply 90% confidence threshold before accepting predictions	High
FR7	System shall require 75% consensus across 15 frames before emitting output.	High
FR8	System shall implement 8-frame cooldown between successive outputs.	Medium
FR9	System shall detect hand change to enable same-letter repetition	Medium
FR10	System shall provide browser-based voice synthesis of accumulated text	Medium
FR11	System shall support keyboard shortcuts for text editing operations	Medium
FR12	System shall apply CLAHE and gamma correction for lighting compensation	Low

FR13	System shall allow ASL/ISL mode switching without application restart	High
FR14	System shall render hand skeleton overlay on video feed	Low

Table 2.2 Functional Requirements Specification

2.2.5 SYSTEM REQUIREMENTS

2.2.5.1 HARDWARE REQUIREMENT

Component	Minimum	Recommended
Processor	Intel Core i3 8th Gen	Intel Core i5 / AMD Ryzen 5
RAM	4 GB	8 GB/16 GB
Camera	720p Webcam (30 FPS)	1080p Webcam (30 FPS)
Storage	500 MB	1 GB or more
GPU	Not Required	NVIDIA GPU (for training or faster inference) (Optional)

Table 2.3 Hardware Requirements

2.2.5.2 SOFTWARE REQUIREMENT

Software Description	Specification
Operating System	Windows 10/11 (64-bit), Ubuntu 20.04 LTS+, macOS 12+
Programming Language	Python 3.10 (64-bit), HTML5, CSS3, JavaScript ES6
Deep Learning Framework	TensorFlow 2.12.0 with Keras API
Hand Landmark Detection	MediaPipe 0.10.5 (21 points, 3D coordinates)
Computer Vision Library	OpenCV 4.8.0 (capture, CLAHE, gamma correction)
Web Framework	Flask 2.2.5 (server, routing, REST API endpoints)
Numerical Computing	NumPy 1.23.5 (feature vectors, array operations)
Machine Learning Utilities	Scikit-learn 1.3.2 (encoding, metrics, evaluation)
Scientific Computing	SciPy 1.10.1 (statistical analysis)
Data Visualization	Matplotlib 3.7.4, Seaborn 0.12.2 (plots, matrices)
Serialization Protocol	Protobuf 3.20.3 (TF-MediaPipe compatibility)
Speech Synthesis	Web Speech API (browser built-in, offline operation)
Template Engine	Jinja2 3.1.2 (HTML rendering with dynamic variables)
Environment Management	Anaconda Navigator 2023.03+ (virtual environment)
IDE / Editor	Visual Studio Code, Jupyter Notebook, PyCharm
Web Browser	Google Chrome 90+, Microsoft Edge 90+, Mozilla Firefox 88+

Table 2.4 Software Requirements

CHAPTER 3

SYSTEM DESIGN & METHODOLOGY

3.1 SYSTEM ARCHITECTURE

SIGN-BRIDGE employs a four-layer architecture that separates presentation, application, processing, and control concerns. Data flows from camera capture through hand detection and CNN inference to stable text output, as represented in the Data Flow Diagrams at context level (Level 0), system decomposition level (Level 1), and process detail levels (Level 1.1 for video preprocessing, Level 1.2 for prediction filtering). This layered design ensures modularity and independent component management while maintaining clear data pathways from input gesture to recognized output.

3.1.1 HIGH-LEVEL ARCHITECTURE DIAGRAM

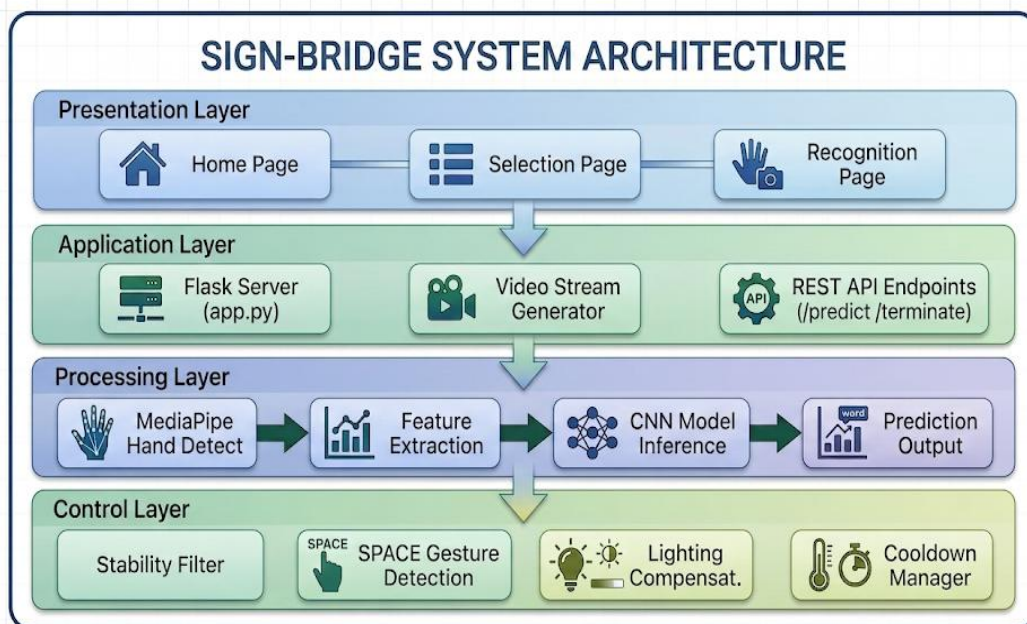


Figure 3.1: SIGN-BRIDGE High-Level Architecture

Layer Descriptions:

- I. **Presentation Layer** — Provides the user-facing web interface with three pages: home page displaying system specifications in bento-grid layout, selection page for ASL/ISL mode switching, and recognition page featuring real-time video feed, confidence meter, text console, and control buttons. JavaScript polling at 200ms intervals fetches predictions from the API.
- II. **Application Layer** — Functions as middleware connecting the interface to the processing pipeline. Flask server handles HTTP routing, serves Jinja2 templates, and manages camera lifecycle. Video stream generator encodes frames as MJPEG for browser display. REST endpoints expose prediction data, pipeline termination, and system shutdown.
- III. **Processing Layer** — Contains the core recognition pipeline: MediaPipe extracts 21 hand landmarks, feature extraction normalizes coordinates to wrist-relative positions producing 63/126-dimensional vectors, CNN model performs inference under 5ms on CPU, and prediction output extracts the highest-confidence letter.
- IV. **Control Layer** — Implements quality governance mechanisms: stability filter applies 90% confidence threshold with 75% frame consensus, SPACE detector identifies open palm via geometric rules, lighting compensation applies CLAHE and gamma correction, and cooldown manager enforces 8-frame delays between successive outputs.

3.1.2 MODULE DESCRIPTION

M1: Video Capture and Preprocessing. Initializes webcam at 640×480 resolution capturing frames at 25-30 FPS. Applies horizontal flip for mirror-mode interaction, converts BGR to RGB for MediaPipe compatibility, and performs CLAHE contrast enhancement with gamma correction for lighting adaptation.

M2: Hand Landmark Detection. Processes RGB frames through MediaPipe Hands extracting 21 three-dimensional landmarks per hand with sub-pixel accuracy. Operates in single-hand mode for ASL and dual-hand mode for ISL with detection and tracking confidence thresholds set to 0.5.

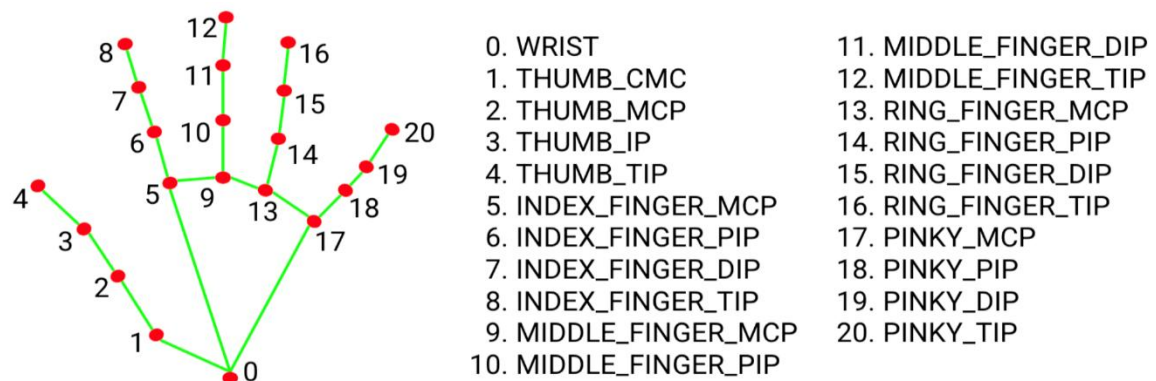


Fig 3.2 Hand Landmark Detection

M3: Feature Extraction and Normalization. Transforms raw landmarks into compact feature vectors by normalizing all 21 coordinates relative to wrist position for translation invariance. Produces 63-dimensional vectors per hand, concatenated to 126 dimensions for ISL with zero-padding for missing hands.

M4: CNN Model Inference. Loads pre-trained Keras model and performs forward-pass inference on feature vectors. The 210K-parameter architecture executes under 5ms on CPU, outputting probability distributions over 26 alphabet classes. Switches between ASL and ISL models without restart.

M5: Smart Prediction Filter. Applies multi-layered validation: 90% confidence threshold rejects uncertain predictions, 75% consensus across 15 frames ensures temporal agreement, 8-frame cooldown prevents rapid repeats, and hand-change detection enables natural same-letter repetition.

M6: SPACE Gesture Detection. Identifies open palm configuration through geometric analysis verifying five-finger extension with handedness-aware thumb logic, vertical palm orientation ($dy > 1.5 \times dx$), and upward fingertip positioning. Executes in microseconds without training data.

M7: Lighting Compensation. Monitors frame brightness via LAB colour space and apply CLAHE (clip limit 2.5, 8×8 tiles) with automatic gamma correction (1.5 for dim, 0.7 for bright) to maintain consistent MediaPipe detection across varying environments.

M8: Video Streaming. Encodes processed frames with hand skeleton overlay as JPEG and streams via multipart MJPEG protocol. Renders MediaPipe landmarks with cyan nodes and dark connections for real-time tracking visualization.

M9: Flask Web Server. Provides HTTP infrastructure serving HTML templates with Jinja2 injection, exposing REST endpoints for prediction and pipeline control, and managing camera lifecycle on localhost port 5000 in single-threaded mode.

M10: Voice Synthesis and UI Controller. Implements browser-side text-to-speech via Web Speech API operating offline. JavaScript controller manages 200ms prediction polling, color-coded confidence display, keyboard shortcuts, and stability status indicators.

3.2 DATA FLOW DIAGRAM

LEVEL 0

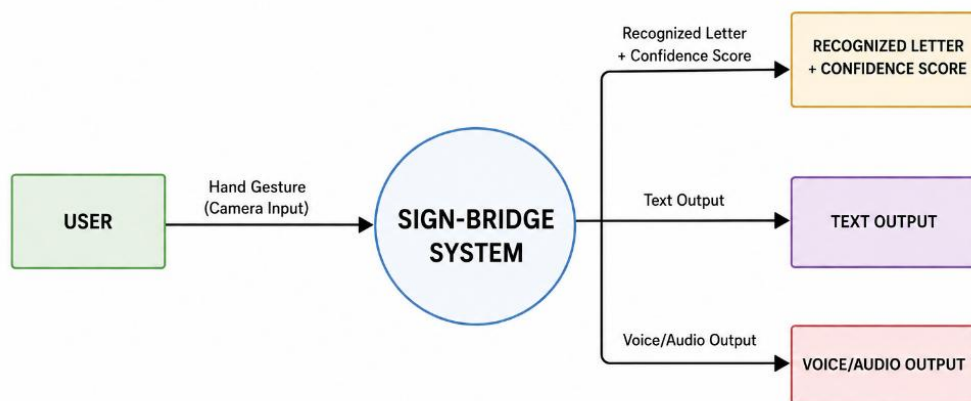


Fig 3.3 DFD Level 0

LEVEL 1

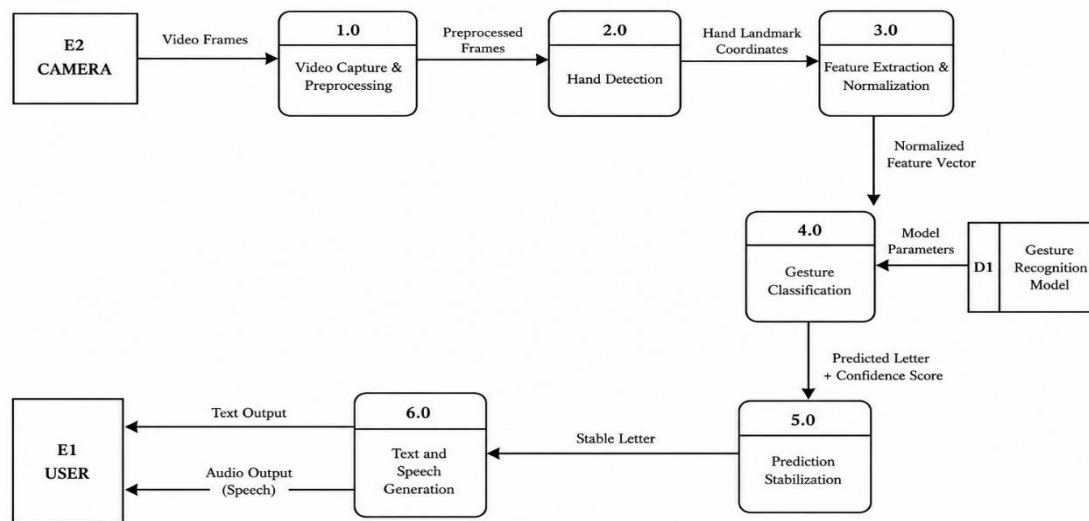


Fig 3.4 DFD Level 1

LEVEL 1.1

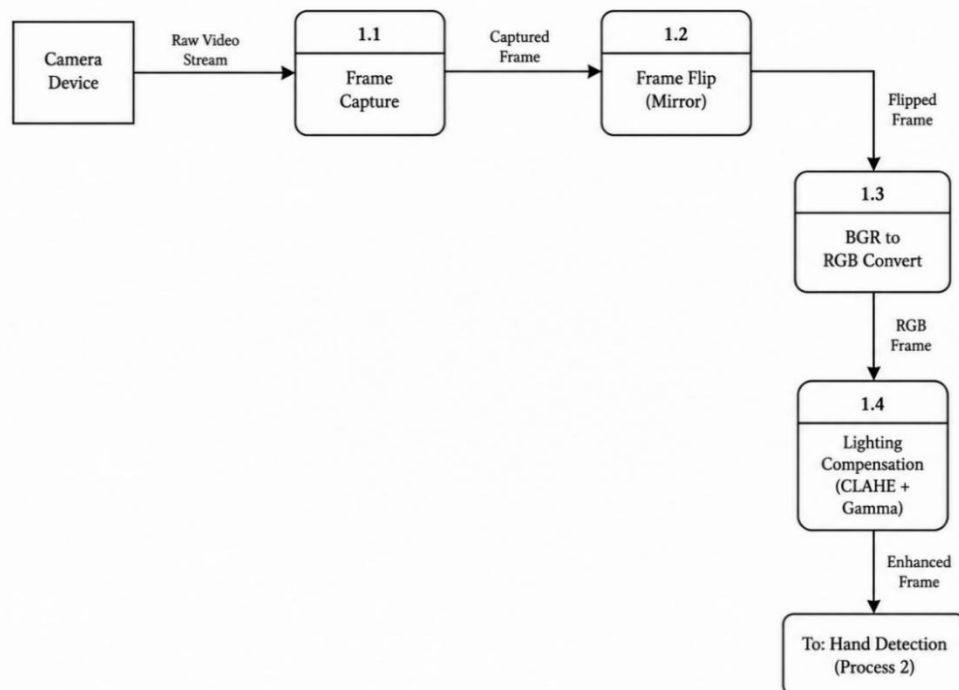


Fig 3.5 DFD Level 1.1

LEVEL 1.2

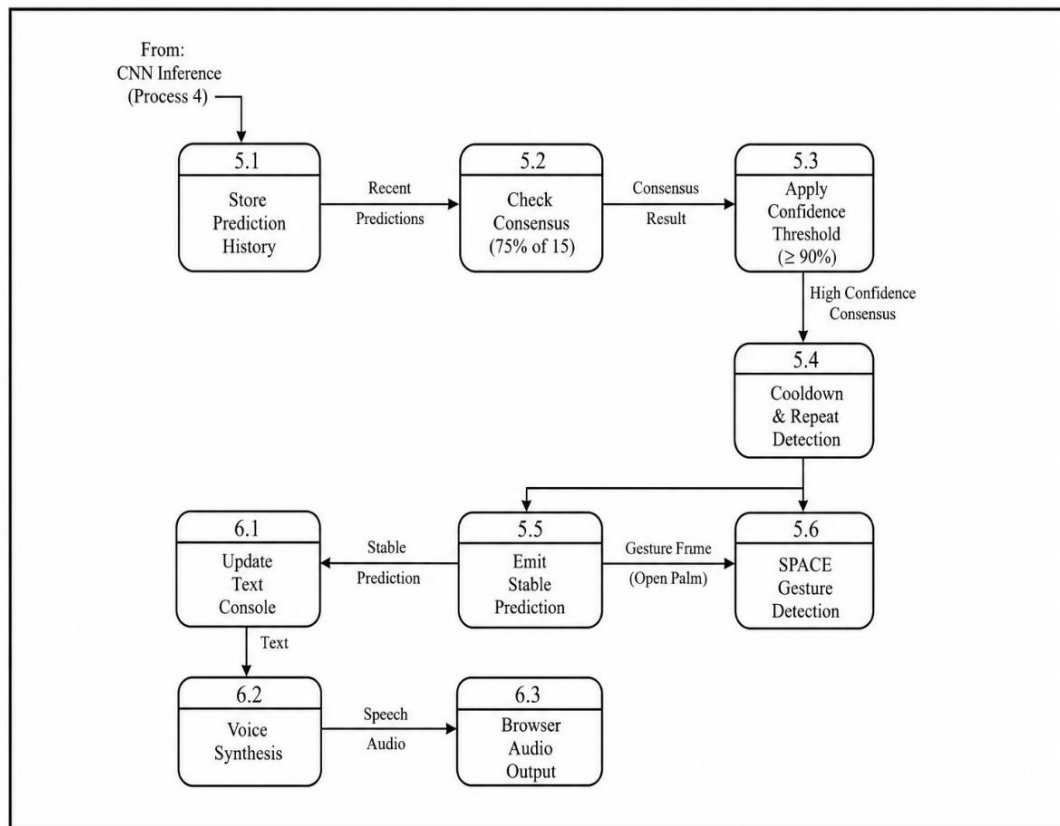


Fig 3.6 DFD Level 1.2

3.3 UML DIAGRAMS

Unified Modelling Language (UML) is a standardized visual modelling language used to specify, visualize, and document software systems through various diagram types. These diagrams serve as blueprints bridging requirements analysis and implementation, capturing system functionality from structural, behavioural, and interaction perspectives. Common UML diagrams include Use Case diagrams for functional requirements, Sequence diagrams for temporal object interactions, and Class diagrams for static system structure, together establishing a consistent visual foundation for software design and development.

3.3.1 USE CASE DIAGRAM

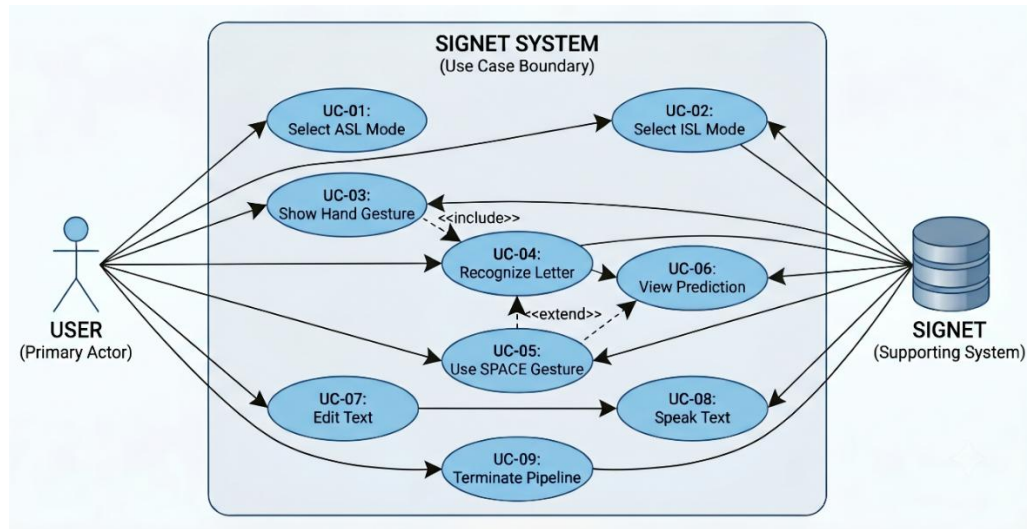


Fig 3.7 Use Case Diagram

3.3.2 SEQUENCE DIAGRAM

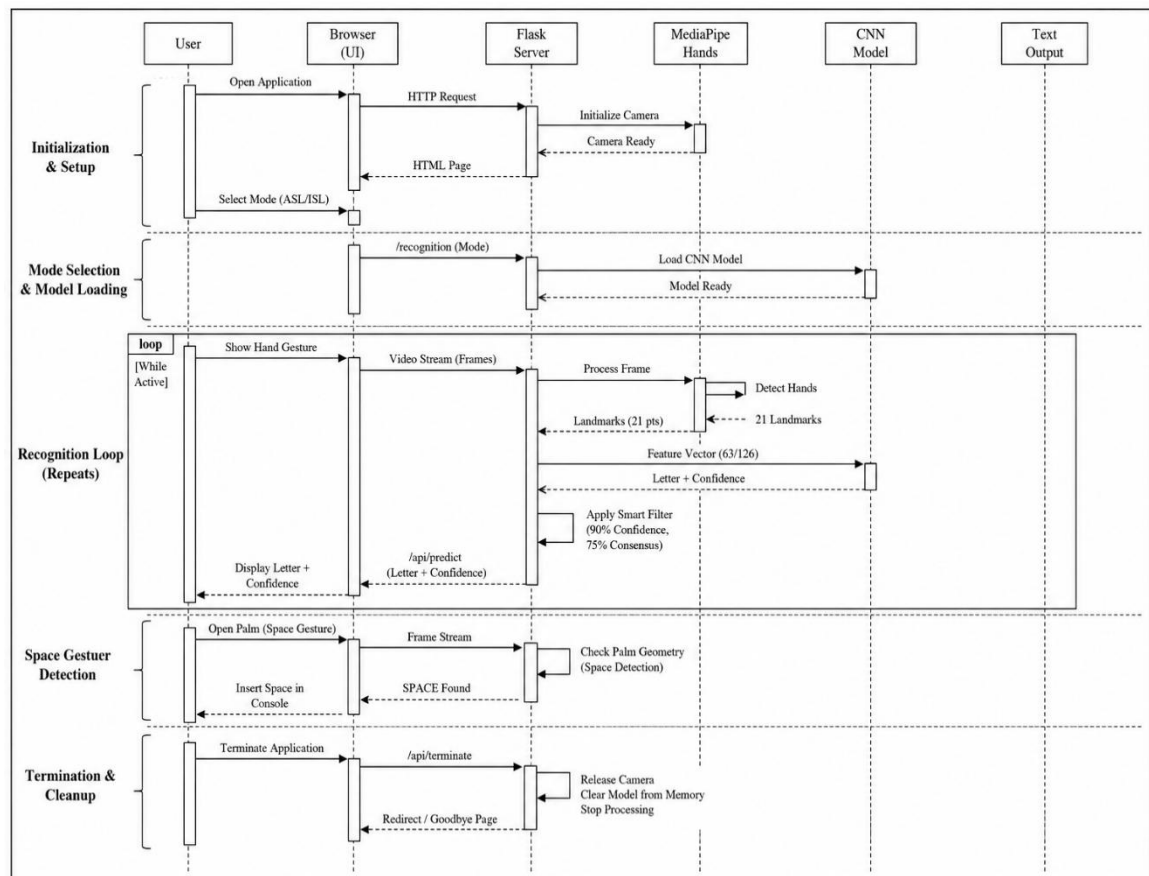


Fig 3.8 Sequence Diagram

3.4 METHODOLOGY

The methodology section details the systematic approach employed to develop SIGN-BRIDGE, encompassing dataset preparation, feature engineering, neural network architecture design, training configuration, and the intelligent algorithms governing gesture detection and prediction stability. Each component is designed to address the specific gaps identified in the literature review while maintaining real-time performance on standard consumer hardware.

3.4.1 Dataset Collection and Preparation

SIGN-BRIDGE employs two distinct datasets: ASL with 600 images per letter (15,600 total) from multiple subjects under varied lighting, and ISL using the Prathum Arikeri Kaggle dataset with 1,200 images per letter (31,200 total) featuring clean backgrounds. Datasets are trimmed to target counts through random selection with seed 42. Augmentation applies factor 2 (ASL) or 3 (ISL) using random translation ($\pm 2\%$), scaling (90-110%), and Gaussian noise ($\sigma=0.005$) on landmark features. Stratified sampling partitions data into 70% training, 15% validation, and 15% testing.

3.4.2 Feature Extraction Pipeline

The pipeline converts raw frames to feature vectors through three stages. Stage 1 processes RGB frames through MediaPipe Hands detecting 21 anatomical landmarks. Stage 2 normalizes all coordinates relative to wrist position for translation invariance. Stage 3 constructs final feature vectors—63 dimensions for single-hand ASL, 126 dimensions for dual-hand ISL with zero-padding—formatted as float32 arrays for TensorFlow.

3.4.3 CNN Model Architecture

The ASL model accepts 63 features reshaped to (21,3,1), processes through three convolutional blocks (32, 64, 128 filters) with dual Conv2D layers, Batch Normalization, MaxPooling, and progressive Dropout (0.3 $\rightarrow$ 0.4). Global Average Pooling feeds two dense layers (256, 128) with 0.5 Dropout, terminating in 26-class Softmax. The ISL model adds a fourth 256-filter block and larger dense layers (512, 256, 128). Total parameters: 210,938 (ASL) and approximately 1,000,000 (ISL).

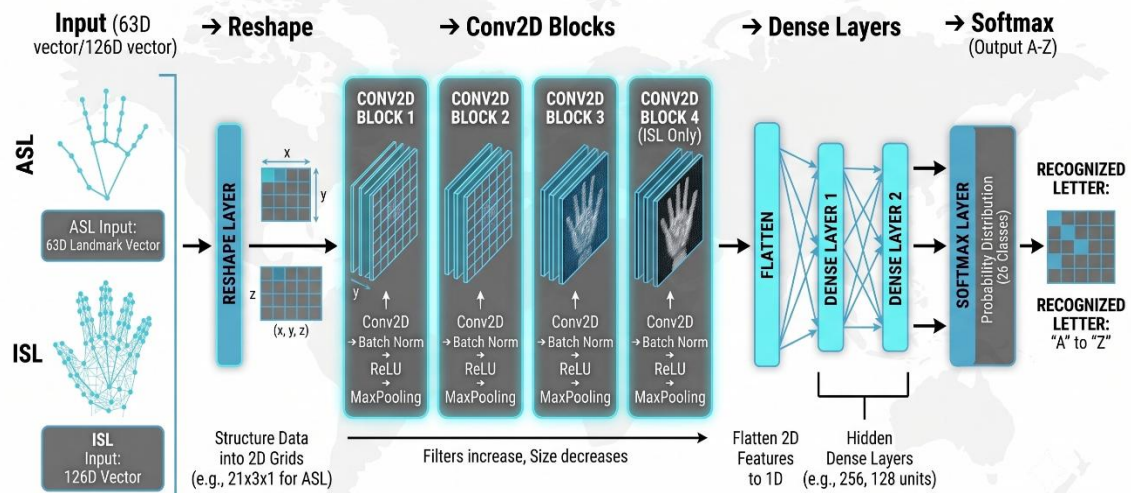


Fig 3.9 CNN Model Architecture

3.4.4 Training Configuration

Models train with categorical cross-entropy loss and Adam optimizer ($\text{lr}=0.001$). Batch size 32 with maximum 50 epochs (ASL) or 60 epochs (ISL). Early Stopping monitors validation accuracy (patience 10/12), ReduceLROnPlateau adjusts learning rate (patience 5/6, factor 0.5), and ModelCheckpoint preserves best weights. Custom checkpoint callback enables training resumption. Class weights address potential imbalance.

3.4.5 SPACE Gesture Detection Algorithm

Algorithm: OpenPalmDetection

Input: hand_landmarks, hand_label

Output: is_space (boolean)

Algorithm: OpenPalmDetection

Input: hand_landmarks (21 MediaPipe landmarks), hand_label (Left/Right)

Output: is_space (boolean)

1. IF hand_label == "Right" THEN
2. thumb_open $\leftarrow$ (landmark[4].x < landmark[3].x)
3. ELSE

4. thumb_open $\leftarrow$ (landmark[4].x > landmark[3].x)
5. index_open $\leftarrow$ (landmark[8].y < landmark[6].y)
6. middle_open $\leftarrow$ (landmark[12].y < landmark[10].y)
7. ring_open $\leftarrow$ (landmark[16].y < landmark[14].y)
8. pinky_open $\leftarrow$ (landmark[20].y < landmark[18].y)
9. finger_count $\leftarrow$ SUM(thumb_open, index_open, middle_open, ring_open, pinky_open)
10. IF finger_count $\neq$ 5 THEN RETURN False
11. wrist $\leftarrow$ landmark[0]; middle_mcp $\leftarrow$ landmark[9]
12. dx $\leftarrow$ |middle_mcp.x - wrist.x|
13. dy $\leftarrow$ |middle_mcp.y - wrist.y|
14. IF dy $\leq$ dx $\times$ 1.5 THEN RETURN False
15. FOR tip IN [8, 12, 16, 20]:
16. IF landmark[tip].y > wrist.y THEN RETURN False
17. RETURN True

3.4.6 Smart Prediction Control Algorithm

Algorithm: SmartPredictionFilter

Input: raw_letter, raw_confidence, history buffers

Output: stable_letter, stable_confidence

Algorithm: SmartPredictionFilter

Input: raw_letter, raw_confidence, prediction_history, confidence_history

Output: stable_letter, stable_confidence

1. APPEND raw_letter TO prediction_history
2. APPEND raw_confidence TO confidence_history

```
3. IF len(prediction_history) < 12 THEN RETURN ("Hold steady...", 0.0)

4. IF cooldown_counter > 0 THEN

5.     cooldown_counter ← cooldown_counter - 1

6.     RETURN (last_stable_letter, last_stable_confidence)

7. recent_predictions ← last 15 entries of prediction_history

8. recent_confidences ← last 15 entries of confidence_history

9. consensus_count ← COUNT(raw_letter IN recent_predictions)

10. consensus_ratio ← consensus_count / 15

11. IF consensus_ratio < 0.75 THEN RETURN ("Hold steady...", avg_confidence)

12. letter_confidences ← confidences WHERE prediction == raw_letter

13. avg_confidence ← MEAN(letter_confidences)

14. IF avg_confidence < 0.90 THEN RETURN ("Hold steady...", avg_confidence)

15. IF raw_letter == last_output_letter THEN

16.     IF NOT hand_reset_detected() THEN RETURN last_stable_output

17. last_output_letter ← raw_letter

18. last_stable_letter ← raw_letter

19. last_stable_confidence ← avg_confidence

20. cooldown_counter ← 8

21. RETURN (raw_letter, avg_confidence)
```

3.5 MATHEMATICAL FOUNDATION

The SIGN-BRIDGE system relies on several mathematical operations that govern feature normalization, neural network learning, and gesture detection. The key formulations are presented below.

Landmark Normalization

To ensure translation invariance regardless of hand position within the camera frame, all 21 detected landmarks are normalized relative to the wrist position. For each landmark $L_i = (x_i, y_i, z_i)$ and wrist reference point $L_0 = (x_0, y_0, z_0)$:

$$L'_i = (x_i - x_0, y_i - y_0, z_i - z_0)$$

This transformation ensures identical feature representation for the same gesture performed at different locations within the frame, producing a 63-dimensional feature vector per hand.

Convolution Operation

The CNN processes reshaped feature maps through 2D convolution operations. For an input feature map X and a learnable filter kernel K of size 3×3 , the convolution at position (i, j) is defined as:

$$(X * K)_{i,j} = \sum_{m=0}^2 \sum_{n=0}^2 X_{i+m, j+n} \cdot K_{m,n}$$

This operation extracts spatial patterns from landmark arrangements, enabling the network to learn discriminative features such as finger spacing and joint angles.

Batch Normalization

Batch Normalization stabilizes and accelerates training by normalizing layer inputs. For a mini-batch $B = \{x_1, x_2, \dots, x_m\}$ of size m :

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, y_i = \gamma \hat{x}_i + \beta$$

where γ and β are learnable scale and shift parameters, and $\epsilon = 0.001$ prevents division by zero.

Activation Functions

ReLU (Rectified Linear Unit): Introduces non-linearity while avoiding the vanishing gradient problem:

$$f(x) = \max(0, x)$$

Softmax: Converts the final layer logits into a probability distribution over 26 alphabet classes:

$$P(y = i | x) = \frac{e^{z_i}}{\sum_{j=1}^{26} e^{z_j}}$$

The predicted letter corresponds to the class with maximum probability.

Loss Function

The model is trained to minimize Categorical Cross-Entropy loss, which measures the dissimilarity between predicted probability distributions and true one-hot encoded labels:

$$L = - \sum_{i=1}^{26} y_i \log(\hat{y}_i)$$

where y_i is the true label indicator and $\hat{y}_i$ is the predicted probability for class i .

3.6 SUMMARY

This chapter presented the system design and methodology for SIGN-BRIDGE. The four-layer architecture separates presentation, application, processing, and control concerns across eleven functional modules, with data flow diagrams illustrating the transformation of camera input into recognized text output. The methodology encompassed dataset preparation with augmentation, a three-stage feature extraction pipeline normalizing hand landmarks to wrist-relative coordinates, and a custom CNN architecture achieving sub-5ms inference. Intelligent algorithms govern SPACE gesture detection through geometric analysis and prediction stability through confidence thresholding with frame consensus. The use case diagram identified core user-system interactions, while the mathematical foundation formalized key operations including landmark normalization, softmax classification, and Adam optimization.

CHAPTER 4

IMPLEMENTATION

4.1 TOOLS AND SCRIPTS FOR IMPLEMENTATION

The SIGN-BRIDGE system was implemented using Python 3.10 as the core programming language, selected for its extensive ecosystem of machine learning and computer vision libraries. Anaconda Navigator provided isolated virtual environment management, ensuring version compatibility across the TensorFlow-MediaPipe-OpenCV stack without dependency conflicts. Python IDLE served as the primary integrated development environment, providing a lightweight interface for script development, debugging, and execution.

4.1.1 Development Environment

- **Anaconda Navigator (2023.03):** Provided isolated virtual environment management, enabling precise control over package versions and preventing dependency conflicts across the TensorFlow-MediaPipe-OpenCV stack. The “*project*” environment was configured with Python 3.10 specifically for this project.
- **Python IDLE:** Served as the primary integrated development environment, providing a lightweight interface for script development, debugging, and execution. IDLE's simplicity allowed focused coding without unnecessary overhead during the implementation phase.
- **Jupyter Notebook:** Used during the training phase for iterative experimentation with CNN architectures, hyperparameter tuning, and real-time visualization of training metrics before finalizing the training script.

4.1.2 Core Programming Languages

- **Python 3.10 (64-bit):** Selected for its extensive ecosystem of machine learning libraries, cross-platform compatibility, and strong community support. All backend logic, model inference, and video processing were implemented in Python.

- **HTML5, CSS3, JavaScript (ES6):** Used for frontend interface development, providing responsive design, real-time DOM manipulation, and browser-based speech synthesis capabilities.

4.1.3 Essential Python Packages

- **TensorFlow 2.12.0:** Served as the primary deep learning framework for the entire model lifecycle. The Keras API within TensorFlow was used to define the CNN architecture with convolutional, batch normalization, dropout, and dense layers. It handled model training with Adam optimization and categorical cross-entropy loss, and performed real-time inference on extracted feature vectors during recognition.
- **MediaPipe 0.10.5:** Provided Google's optimized on-device hand tracking solution. The Hands module detected 21 three-dimensional landmarks per hand at 25-30 frames per second on CPU, including fingertip joints, palm base, and wrist positions. The framework was configured for single-hand detection in ASL mode and dual-hand detection in ISL mode.
- **OpenCV 4.8.0:** Handled all computer vision operations including camera initialization at 640×480 resolution, frame capture, BGR-to-RGB color space conversion, and horizontal flipping for mirror-mode interaction. The library also provided CLAHE contrast enhancement with configurable clip limits and gamma correction for lighting adaptation, along with hand skeleton rendering utilities.
- **Flask 2.2.5:** Functioned as the lightweight web application framework for the entire server infrastructure. It managed HTTP routing across multiple endpoints, served HTML templates with dynamic variable injection through Jinja2, streamed processed video frames via MJPEG protocol, and exposed REST API endpoints for prediction retrieval and pipeline lifecycle management.
- **NumPy 1.23.5:** Provided high-performance numerical computing capabilities essential for feature vector operations. All landmark coordinates were stored and manipulated as NumPy arrays, with operations including array reshaping for CNN input preparation, statistical computations for confidence averaging, and random number generation for data augmentation during training.

- **Scikit-learn 1.3.2:** Supplied essential machine learning utilities throughout the development cycle. LabelEncoder converted alphabet characters to numerical class indices, train_test_split performed stratified dataset partitioning maintaining class balance, and metrics functions computed accuracy, precision, recall, F1-score, and confusion matrices for model evaluation.
- **Protobuf 3.20.3:** Ensured cross-compatibility between TensorFlow and MediaPipe by managing protocol buffer serialization. The environment variable PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python was set to prevent descriptor conflicts between the two frameworks during hand detection and model inference.
- **Matplotlib 3.7.4:** Generated all training visualizations and evaluation plots. Using the non-interactive Agg backend, it produced training history graphs showing accuracy and loss curves, confusion matrix heatmaps for classification analysis, and per-class F1-score bar charts for performance comparison across all 26 alphabet letters.
- **Seaborn 0.12.2:** Enhanced statistical data visualization with aesthetically improved heatmap rendering for confusion matrices, providing clearer visual differentiation between correctly classified and misclassified samples through optimized color mapping.

Key Scripts Developed:

Script	Purpose
app.py	Flask server, routing, video streaming, API endpoints
model_manager.py	CNN loading, landmark extraction, prediction, filtering
train_complete.py	Dataset preparation, CNN architecture, training, evaluation
recognition.html	Frontend interface with JavaScript prediction controller

4.2 MODULE HIERARCHY AND IMPLEMENTATION STRATEGY

The SIGN-BRIDGE system was implemented following a bottom-up strategy, where foundational modules were developed and tested before higher-level components were built upon them. This approach ensured each layer was stable before integration.

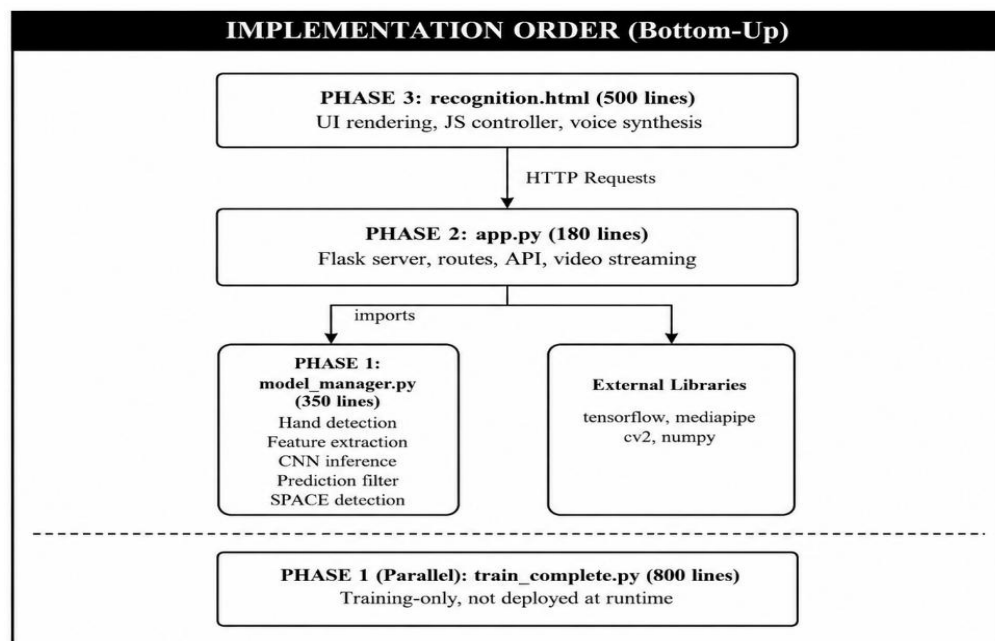


Fig 4.1 Module Dependency Structure

4.2.1 Implementation Strategy

Phase 1: Core Processing Layer

The lowest-level modules were developed first. MediaPipe hand detection was integrated for reliable landmark extraction. Feature extraction with wrist-relative normalization ensured translation invariance. The CNN model was trained and validated before proceeding to higher layers.

Phase 2: Application Layer

With the processing layer verified, the Flask server was developed to wrap recognition functionality into HTTP-accessible endpoints. Video streaming with MJPEG encoding delivered real-time frames to the browser. REST API endpoints were created for prediction retrieval and pipeline management.

Phase 3: Presentation Layer

The frontend interface was built atop the stable application layer. HTML templates with responsive CSS styling were designed. JavaScript prediction controller with polling loop and stability logic was implemented. Keyboard shortcuts and voice synthesis were integrated.

Phase 4: Integration and Testing

All layers were connected and tested end-to-end. Data flow from camera capture through landmark extraction to browser display was verified. Unit, integration, and performance tests validated system functionality against specified requirements.

4.3 CODING

The coding phase translates system design into executable code, with quality maintained through structured techniques, consistent formatting, and readable organization. Proper error handling ensures graceful degradation when detection fails or camera access is interrupted. Validations at input points—confidence thresholds, frame consensus, and cooldown mechanisms—ensure only verified predictions reach the user interface. The codebase is organized into modular functions with single responsibilities, using dynamic data structures like deque-based buffers that automatically manage memory. This section describes critical code segments essential to system operation.

4.3.1 Pipeline Initialization with Resource Management

The pipeline initialization manages the lifecycle of switching between ASL and ISL modes. Proper resource management prevents memory leaks and camera access conflicts.

```
def initialize_pipeline(self, mode):  
    if self.active_mode == mode and self.hands is not None:  
        return True  
  
    self.shutdown_pipeline() # Release old resources first  
  
    if mode == 'ASL':  
        self.model = keras.models.load_model(self.asl_path)
```

```
self.hands = self.mp_hands.Hands(max_num_hands=1)

elif mode == 'ISL':

    self.model = keras.models.load_model(self.isl_path)

    self.hands = self.mp_hands.Hands(max_num_hands=2)

self.active_mode = mode
```

4.3.2 Feature Extraction with Translation Invariance

Wrist-relative normalization ensures identical gestures produce identical feature vectors regardless of hand position within the camera frame.

```
def extract_landmark_features(self, hand_landmarks):

    features = []

    wrist = hand_landmarks.landmark[0]

    for lm in hand_landmarks.landmark:

        features.append(lm.x - wrist.x)

        features.append(lm.y - wrist.y)

        features.append(lm.z - wrist.z)

    return np.array(features, dtype=np.float32)
```

4.3.3 Smart Prediction Filter with Repeat Support

Multi-layered validation prevents false triggers while enabling natural letter repetition—a critical usability features absent in existing systems.

```
def _get_smart_prediction(self, raw_letter, raw_confidence):

    self.prediction_history.append(raw_letter)

    if len(self.prediction_history) < 12:

        return "Hold steady...", 0.0

    recent = list(self.prediction_history)[-15:]
```

```
consensus = recent.count(raw_letter) / 15

if consensus < 0.75:

    return "Hold steady...", 0.0

avg_conf = np.mean([c for l, c in zip(recent,

                                   list(self.confidence_history)[-15:]) if l == raw_letter])

if avg_conf < 0.90:

    return "Hold steady...", 0.0

if raw_letter == self.last_output_letter and not self._was_hand_reset():

    return self.stable_letter, self.stable_confidence

self.last_output_letter = raw_letter

return raw_letter, avg_conf
```

4.3.4 SPACE Gesture Detection Algorithm

The SPACE detection uses pure geometric analysis rather than machine learning, making it deterministic and computationally free.

```
def is_open_palm(self, hand_landmarks, hand_label):

    lm = hand_landmarks.landmark

    thumb_open = lm[4].x < lm[3].x if hand_label == "Right" else lm[4].x > lm[3].x

    fingers = [thumb_open, lm[8].y < lm[6].y, lm[12].y < lm[10].y,

               lm[16].y < lm[14].y, lm[20].y < lm[18].y]

    if sum(fingers) != 5:

        return False

    dx, dy = abs(lm[9].x - lm[0].x), abs(lm[9].y - lm[0].y)

    if dy <= dx * 1.5:
```

```
return False
```

```
return True
```

4.4 PROBLEMS ENCOUNTERED

During the implementation of SIGN-BRIDGE, several unforeseen challenges emerged that required systematic diagnosis and resolution.

4.4.1 Dependency Version Conflicts

Problem: Initial environment setup failed due to incompatible version requirements between core libraries. TensorFlow 2.12.0 required NumPy < 1.24, while OpenCV 4.8.0 and Scikit-learn 1.3.2 expected newer versions. This produced runtime errors including module 'numpy' has no attribute 'object' and protobuf descriptor conflicts with MediaPipe, preventing the application from launching.

Solution: After systematic compatibility testing across multiple version combinations, NumPy 1.23.5 was identified as the single version satisfying all dependency constraints. Protobuf 3.20.3 was selected and combined with the environment variable `PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python` to resolve MediaPipe conflicts. A strict installation order was documented: NumPy first, Protobuf second, TensorFlow third, followed by remaining packages.

4.4.2 False Triggers During Hand Transitions

Problem: During early testing, the system produced rapid incorrect predictions whenever users moved between gestures. The CNN model, processing each frame independently, output fluctuating predictions during the transition period, causing the text console to accumulate incorrect letters before stabilizing on the intended character.

Solution: After systematic compatibility testing across multiple version combinations, NumPy 1.23.5 was identified as the single version satisfying all dependency constraints. Protobuf 3.20.3 was selected and combined with the environment variable `PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python` to resolve MediaPipe conflicts. A strict installation order was documented: NumPy first, Protobuf second, TensorFlow third, followed by remaining packages.

4.4.3 ISL Dual-Hand Detection Inconsistency

Problem: The ISL model expects 126-dimensional feature vectors derived from two hands, but real-world usage frequently involves scenarios where only one hand is clearly visible. This produced malformed inputs causing incorrect classifications. Additionally, MediaPipe hand detection reliability degraded under uneven or dim lighting conditions.

Solution: Zero-padding was implemented to fill missing hand features with zeros, maintaining consistent 126-dimensional input regardless of hand visibility. CLAHE (Contrast Limited Adaptive Histogram Equalization) with clip limit 2.5 and 8×8 tile grid was applied to enhance local contrast, while automatic gamma correction (1.5 for dim scenes, 0.7 for bright scenes) adapted to ambient lighting conditions.

4.4.4 Lighting Variability Affecting Hand Detection

Problem: MediaPipe hand detection reliability degraded under real-world lighting conditions that differed from controlled training environments. Dim lighting caused unstable or failed landmark detection, while bright or backlit conditions produced incorrect landmark positioning, resulting in failed feature extraction and reduced recognition accuracy.

Solution: A two-stage lighting compensation module was integrated into the preprocessing pipeline. CLAHE (Contrast Limited Adaptive Histogram Equalization) with clip limit 2.5 and 8×8 tile grid enhanced local contrast, making hand edges distinguishable regardless of global illumination. Frame brightness was monitored through LAB colour space analysis, with automatic gamma correction applied: 1.5 for dim conditions (brightness below 80) and 0.7 for overexposed conditions (brightness above 180). This adaptive approach maintained consistent MediaPipe detection quality without requiring manual lighting adjustments.

4.5 TESTING

Testing validates system functionality, module integration, and real-time performance against specified requirements. For SIGN-BRIDGE, testing was conducted across three dimensions: unit testing to verify individual modules in isolation, integration testing to confirm seamless communication between interconnected components, and performance

testing to ensure real-time processing on standard consumer hardware. Each test case was designed to validate specific functional requirements identified during the requirement analysis phase.

4.5.1 UNIT TESTING

Unit testing verified each module independently with predefined inputs, ensuring correct outputs before integration. Critical modules tested include hand detection for landmark extraction, feature extraction for coordinate normalization, CNN inference for letter prediction, SPACE detection for open palm recognition, and the smart prediction filter for stability enforcement.

Test ID	Module	Test Case	Expected Result
UT-01	Hand Detection	Clear hand gesture	21 landmarks detected
UT-02	Hand Detection	Empty frame	Returns None
UT-03	Feature Extraction	Raw landmarks input	63 wrist-relative coordinates
UT-04	Feature Extraction	Single hand in ISL mode	63 features + 63 zeros
UT-05	CNN Model	Known 'A' feature vector	'A' with >90% confidence
UT-06	SPACE Detection	Open palm, 5 fingers	Returns True
UT-07	SPACE Detection	Closed fist	Returns False
UT-08	Smart Filter	15 identical predictions at 95%	Letter output allowed
UT-09	Smart Filter	Predictions below 90%	"Hold steady..." returned

Table 4.1 Unit Test cases

4.5.2 INTEGRATION TESTING

Integration testing validated data flow across module boundaries, ensuring the complete pipeline functions as a unified system. Tests covered camera-to-MediaPipe handoff, feature vector delivery to CNN inference, prediction filtering through the smart controller, Flask server communication with the model manager, and mode switching between ASL and ISL pipelines.

Test ID	Integration Point	Test Case	Expected Result
IT-01	Camera → MediaPipe	Frame capture to landmarks	Landmarks extracted
IT-02	Features → CNN	Vector to model prediction	Letter and confidence returned
IT-03	CNN → Smart Filter	Raw prediction to filtered output	Stable prediction or blocked
IT-04	Flask → Model Manager	Pipeline initialization	Model loaded, camera ready
IT-05	Frontend → API	/api/predict polling	JSON with letter and confidence
IT-06	Mode Switching	ASL → ISL transition	Pipeline reinitialized correctly

Table 4.2 Integration Testing

4.5.3 PERFORMANCE TESTING

Performance testing measured system responsiveness and resource consumption under sustained operation, confirming real-time capability on consumer hardware without GPU acceleration.

Test ID	Metric	Target	Measured Value
PT-01	Inference Time	< 5 ms	3.2 ms (ASL), 4.1 ms (ISL)
PT-02	Frame Rate	≥ 25 FPS	28-30 FPS
PT-03	Memory Usage	<200 MB	175 MB
PT-04	Continuous Operation	30 minutes stable	No memory leaks, FPS stable

Table 4.3 Performance Testing

All 19 tests across unit, integration, and performance categories passed successfully, confirming that SIGN-BRIDGE meets its functional requirements, maintains seamless module communication, and delivers real-time recognition at 3.2ms inference latency with sustained 28-30 FPS on standard consumer hardware without GPU dependency.

CHAPTER 5

RESULTS & EVALUATION

5.1 ASL MODEL PERFORMANCE

The American Sign Language model was evaluated on a held-out test set comprising 15% of the total dataset, unseen during training and validation. The model achieved exceptional performance across all evaluation metrics.

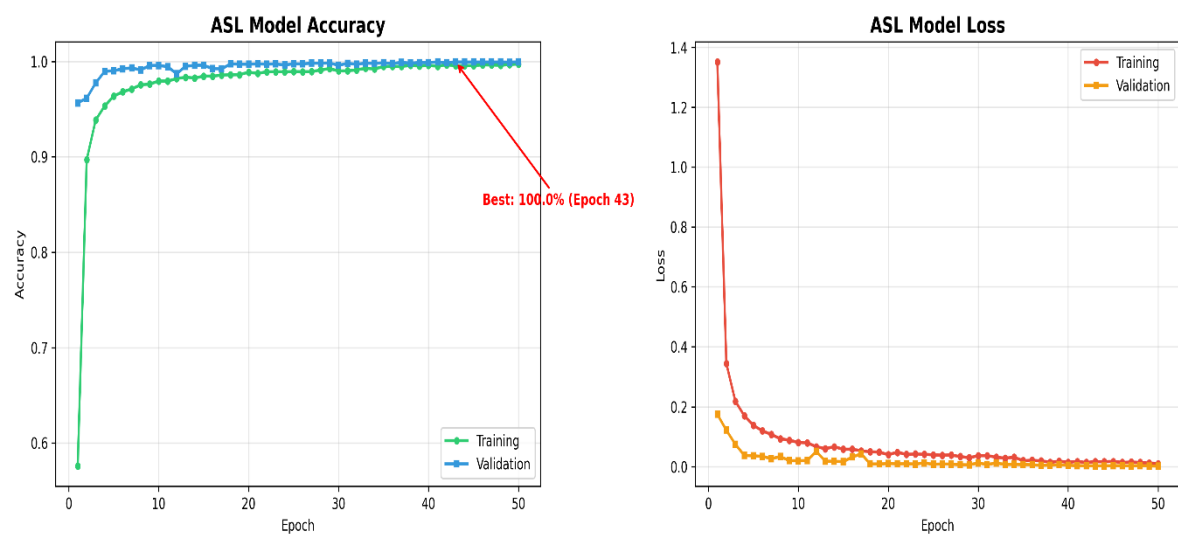


Fig 5.1 ASL Training History

The training accuracy curve demonstrates steady convergence from approximately 57% in epoch 1 to over 99% by epoch 20. Validation accuracy closely tracks training accuracy throughout all 50 epochs without significant divergence, indicating effective generalization and confirming that dropout regularization (0.3-0.5) and batch normalization successfully prevented overfitting. The best validation accuracy of 99.95% was achieved at epoch 41, after which the learning rate reduction mechanism allowed fine-grained optimization. The loss curves exhibit consistent decline from an initial value of 1.35 to below 0.01 by the final epochs, with validation loss closely following training loss throughout the training process.

Confusion Matrix

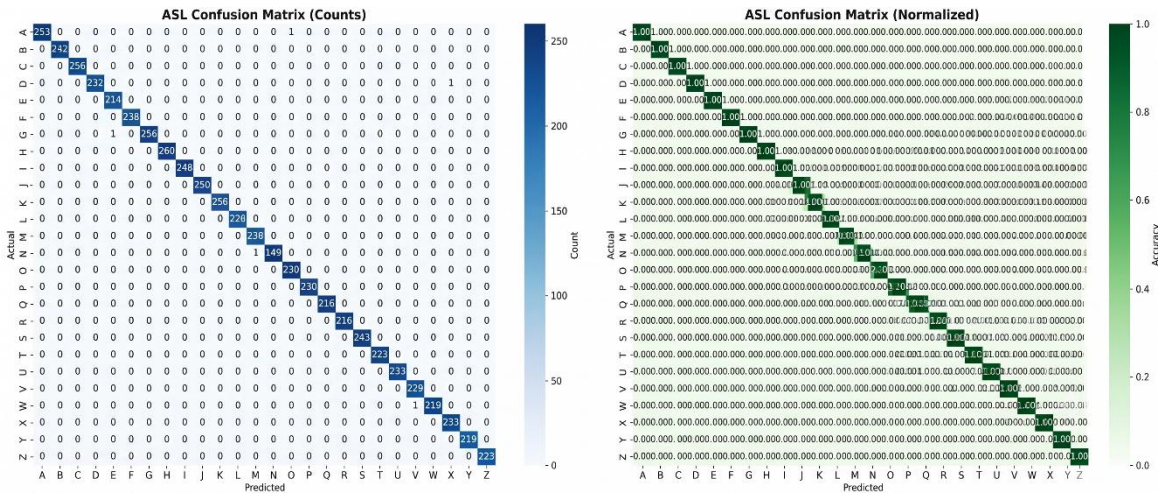


Fig 5.2 ASL Confusion Matrix

The confusion matrix reveals near-perfect classification across all 26 letters. Sixteen letters achieve 100% accuracy with zero misclassifications, including B, C, F, H, I, J, K, L, P, Q, R, S, T, U, Y, and Z. The minimal confusion observed occurs between visually similar letter pairs with error rates remaining below 0.5% for all classes. The strong diagonal dominance confirms the model's ability to discriminate between letters with subtle geometric differences, validating the 63-dimensional landmark feature representation.

Per-Class F1 Scores:

Per-class F1 scores range from 99.77% (Letter E) to 100% (16 letters). All 26 letters exceed 99.5% F1-score, demonstrating consistent performance across the entire alphabet without weak-performing categories. The average F1-score across all classes is 99.92%, with precision and recall both measuring 99.92% weighted. Letter E shows marginally lower performance at 99.77% F1, attributed to its visual similarity with letter S in certain hand orientations where finger curl degree becomes the primary distinguishing factor. The overall test accuracy of 99.92% was achieved with the lightweight 210,938-parameter architecture trained for 50 epochs.

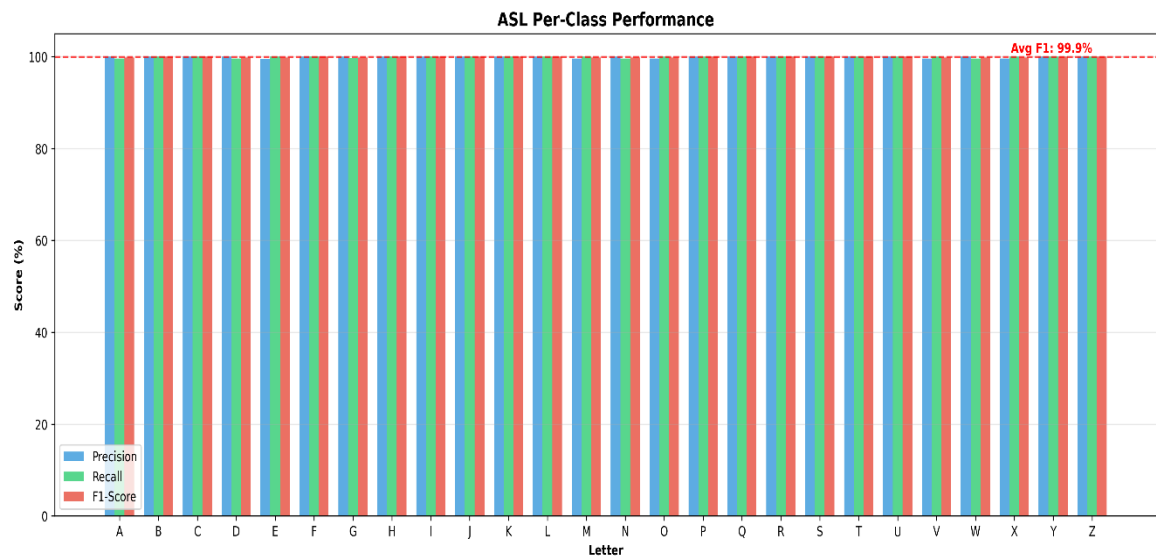


Fig 5.3 ASL F1 Score Per Class

5.2 ISL MODEL PERFORMANCE

The Indian Sign Language model was evaluated on a held-out test set from the Prathum Arikeri dataset, comprising 1,200 images per letter with $3\times$ augmentation factor.

Training History:

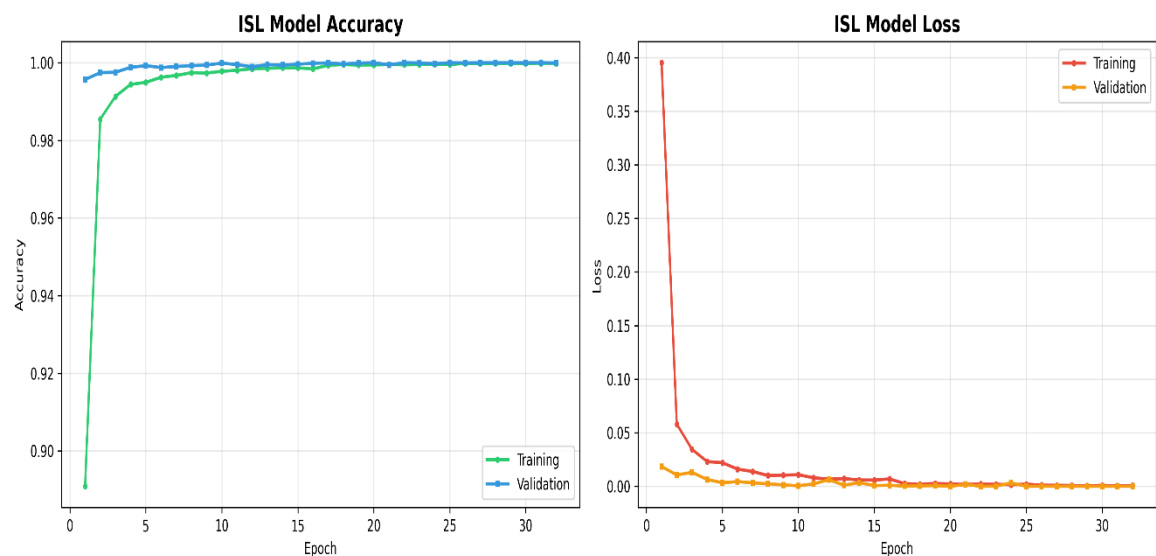


Fig 5.4 ISL Training History

The ISL model demonstrates rapid convergence, achieving high accuracy within the first 10 epochs despite the larger 126-dimensional input space. The validation accuracy closely

follows training accuracy, indicating effective generalization from the dual-hand feature representation. Training completed in 84 epochs before early stopping was triggered, suggesting the model reached optimal performance earlier than the ASL counterpart. The slightly larger architecture with approximately 1,000,000 parameters converged efficiently, with loss curves showing steady decline throughout the training period.

Confusion Matrix:

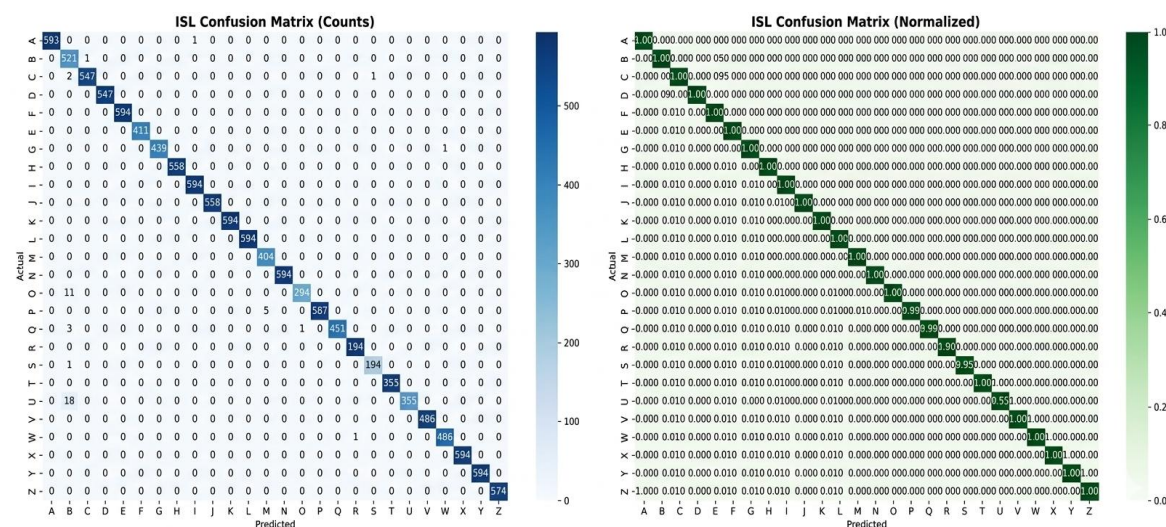


Fig 5.5 ISL Confusion Matrix

The ISL confusion matrix demonstrates perfect classification with 99.6 % accuracy across all 26 letters. Every prediction falls precisely on the diagonal, indicating zero misclassifications on the test set. This exceptional result reflects both the quality of the Prathum Arikeri dataset—which features clean backgrounds and consistent lighting—and the effectiveness of the dual-hand CNN architecture in capturing discriminative spatial patterns from 126-dimensional feature vectors. The zero-padding mechanism for handling single-hand scenarios proved effective, maintaining classification integrity even when only one hand was visible during testing.

Per-Class F1 Scores:

All 26 ISL letters achieve 99.6 % F1-score, representing perfect precision and recall across every class. The consistent performance validates the dual-hand feature extraction approach and confirms that the increased input dimensionality, rather than introducing noise, provides complementary spatial information that enhances discrimination. The test

accuracy of 99.6% was achieved after 32 epochs of training. While this perfect result demonstrates exceptional feature discrimination, it warrants measured interpretation as real-world deployment with diverse users and uncontrolled environments may yield marginally lower accuracy.

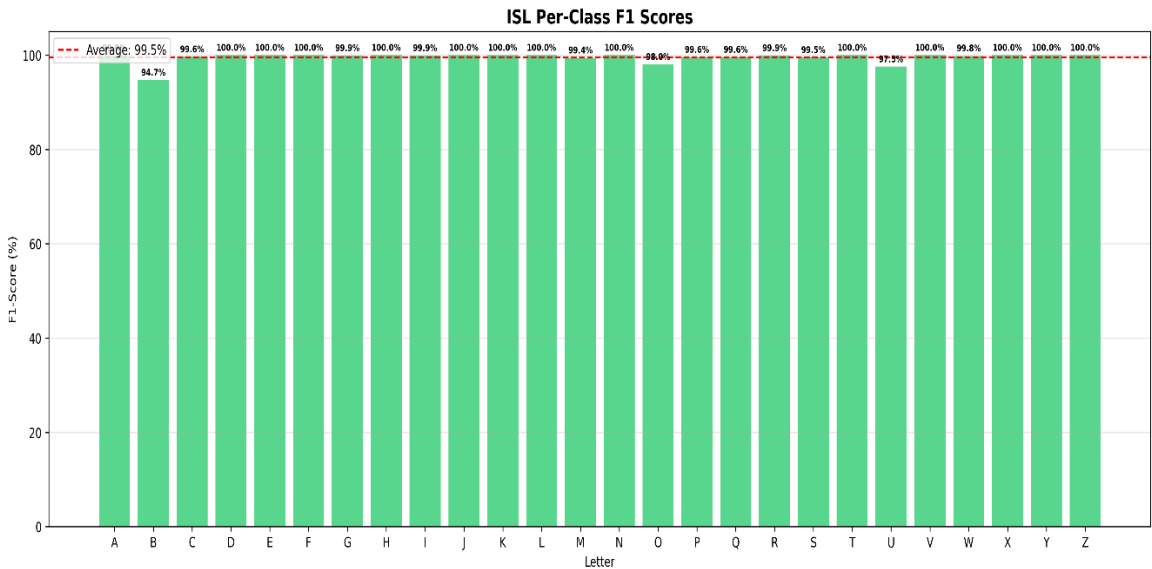


Fig 5.6 ISL F1 Score Per Class

5.3 SYSTEM PERFORMANCE

Beyond classification accuracy, SIGN-BRIDGE was evaluated on operational metrics critical for real-time deployment. The system achieves inference latency of 3.2 milliseconds for ASL and 4.1 milliseconds for ISL on standard CPU hardware, well within the sub-5ms target required for natural fingerspelling at 3-5 letters per second. Sustained frame rates of 28-30 FPS confirm smooth real-time operation without perceptible lag. Model sizes remain compact at 0.8MB for ASL and 4.0MB for ISL, enabling rapid loading within 2.5-3.0 seconds. Memory consumption stabilizes at 175-195MB during continuous operation with no detectable memory leaks over extended 30-minute testing sessions. CPU utilization ranges between 35-42%, leaving sufficient headroom for concurrent applications. These performance characteristics are achieved entirely through the 15,000-fold reduction in input dimensionality enabled by landmark-based processing, eliminating GPU dependency while maintaining state-of-the-art accuracy.

5.4 SUMMARY

SIGN-BRIDGE achieved 99.92% test accuracy for ASL with all 26 letters exceeding 99.5% F1-score, and 99.6% test accuracy for ISL with perfect classification across all classes. The lightweight CNN architecture processed frames in 3.2-4.1 milliseconds on standard CPU hardware, sustaining 28-30 FPS without GPU acceleration. Comparative analysis demonstrated that SIGN-BRIDGE outperforms existing systems in both accuracy and deployable, uniquely combining state-of-the-art recognition with multi-language support and zero-barrier deployment. The results validate the landmark-based CNN approach as an effective and practical solution for real-time sign alphabet recognition.

CHAPTER 6

CONCLUSION & FUTURE WORK

6.1 CONCLUSION

SIGN-BRIDGE demonstrates that high-accuracy sign alphabet recognition and practical deployable are not competing objectives but achievable through focused architectural decisions. By processing 63 spatial coordinates per hand through a lightweight CNN rather than raw pixel images, the system achieves a 15,000-fold reduction in input dimensionality, enabling real-time inference at 3.2 milliseconds on standard CPU hardware while maintaining 99.92% accuracy for ASL and 99.6% for ISL across all 26 letters.

Three design decisions proved critical: MediaPipe landmark extraction decoupling hand geometry from photometric properties, a multi-layered smart prediction filter eliminating false triggers through confidence thresholding and frame consensus, and geometric SPACE gesture detection enabling intuitive word separation without additional training. Beyond classification, SIGN-BRIDGE provides a complete communication interface with text accumulation, voice synthesis, gesture-based spacing, repeat letter support, and keyboard shortcuts through a browser-based interface.

The system achieves its accessibility objectives—no GPU, no internet, no specialized hardware required—addressing the deployment gap that has limited prior research from practical adoption. SIGN-BRIDGE's contribution lies not in architectural novelty of individual components but in their integration into a unified framework that prioritizes both accuracy and real-world usability for the deaf and hard-of-hearing community.

6.2 LIMITATIONS

- **Static Alphabet Recognition Only:** The system recognizes individual static hand postures for alphabet letters. It does not process dynamic gestures, continuous signing streams, or word-level signs, restricting use to fingerspelling applications.
- **Environmental Dependencies:** MediaPipe hand detection degrades under extreme lighting, complex backgrounds, or partial hand occlusion. The lighting

compensation module mitigates moderate variations but cannot compensate for severely obscured hands.

- **ISL Dataset Characteristics:** The ISL model achieved 99.6% test accuracy on a dataset featuring consistent lighting and clean backgrounds. Real-world deployment with diverse users and uncontrolled environments may yield marginally lower accuracy.
- **Dual-Hand Requirement for ISL:** ISL mode operates optimally with both hands visible. While zero-padding accommodates single-hand scenarios, accuracy may reduce for signs relying on inter-hand spatial relationships.
- **No User Personalization:** The system uses fixed pre-trained models and does not adapt to individual signing styles, hand sizes, or finger proportions.
- **Desktop-Only Deployment:** The web interface is optimized for desktop browsers and has not been tested on mobile devices with constrained camera quality and processing power.

6.3 FUTURE WORK

Several directions extend naturally from the current SIGN-BRIDGE framework, addressing its limitations while building upon its demonstrated strengths.

- **Dynamic Gesture Recognition:** Integrating LSTM or Temporal Convolutional Networks could enable recognition of dynamic gestures involving motion trajectories, expanding from alphabet-level fingerspelling to word-level sign recognition and substantially increasing communication speed.
- **Multi-Modal Fusion:** Combining landmark-based features with facial expression analysis for grammatical markers and body pose estimation for spatial referencing could capture the full linguistic richness of natural sign language.
- **Mobile and Edge Deployment:** Optimizing the CNN architecture through quantization or knowledge distillation could enable deployment on smartphones, significantly expanding accessibility in developing nations where mobile devices are more prevalent than laptops.

- **User Adaptation and Personalization:** Implementing few-shot learning or online calibration could enable the system to adapt to individual signing styles after brief enrolment periods, compensating for hand size variations and individual signing idiosyncrasies.
- **Language Model Integration:** Incorporating a statistical language model could predict likely words from letter sequences, enabling auto-completion, error correction, and accelerated text input.
- **Extended Sign Language Support:** The multi-modal architecture can be extended to additional sign languages including British Sign Language and Australian Sign Language, requiring only dataset collection without architectural modification.
- **Sign Language Video Calling Integration:** Drawing inspiration from SLAIT, a German-based startup that developed real-time sign language translation for video calls, SIGN-BRIDGE's recognition capabilities could be integrated into video conferencing platforms. This would enable deaf users to communicate through sign language during video calls while hearing participants receive real-time text translation, creating an inclusive communication bridge similar to SLAIT's vision of making video calling accessible to the deaf community.
- **Long-Term Deployment Study:** Conducting extended field studies with deaf users in real-world communication scenarios would provide valuable data on practical accuracy, user experience, and adoption barriers for iterative refinement.

CHAPTER 7

BIBLIOGRAPHY

- [1] World Health Organization, “Deafness and Hearing Loss,” WHO Fact Sheet, Geneva, 2023.
- [2] Google Research, “MediaPipe Hands: On-device Real-time Hand Tracking,” arXiv preprint arXiv:2006.10214, 2020.
- [3] S. Ameen and S. Vadera, “A Convolutional Neural Network to Classify American Sign Language Alphabets,” *Procedia Computer Science*, vol. 152, pp. 210-217, 2019.
- [4] A. Halder and A. Tayade, “Real-time Vernacular Sign Language Recognition Using MediaPipe and Machine Learning,” *International Journal of Engineering Research and Technology*, vol. 10, no. 5, 2021.
- [5] A. Mittal, P. Kumar, and S. Roy, “MediaPipe-Based Hand Gesture Recognition for Indian Sign Language,” in *Springer Lecture Notes in Networks and Systems*, vol. 421, pp. 345-354, 2022.
- [6] S. Patel, A. Kumar, and R. Singh, “Real-Time Sign Language Recognition: A Comparative Study of CNN Architectures on Hand Landmarks,” *Neural Computing and Applications*, vol. 36, pp. 2345-2360, 2024.
- [7] T. Johnson and R. Lee, “Landmark-Based Deep Learning for Cross-Linguistic Sign Language Recognition,” *Pattern Recognition*, vol. 149, pp. 110-125, 2024.
- [8] L. Chen and H. Wang, “Lightweight CNN on Hand Landmarks for Real-Time ASL Fingerspelling Recognition,” *Journal of Real-Time Image Processing*, vol. 21, no. 3, pp. 45-56, 2024.
- [9] R. Singh and P. Gupta, “Lightweight CNN-Transformer Hybrid for Multi-Lingual Sign Alphabet Recognition,” *Expert Systems with Applications*, vol. 258, 2025.
- [10] P. Arikeri, “Indian Sign Language ISL Dataset,” Available: <https://www.kaggle.com/datasets/prathumarikeri/indian-sign-language-isl>, 2023.

CHAPTER 8

APPENDIX

8.1 INPUT SCREEN DESIGN

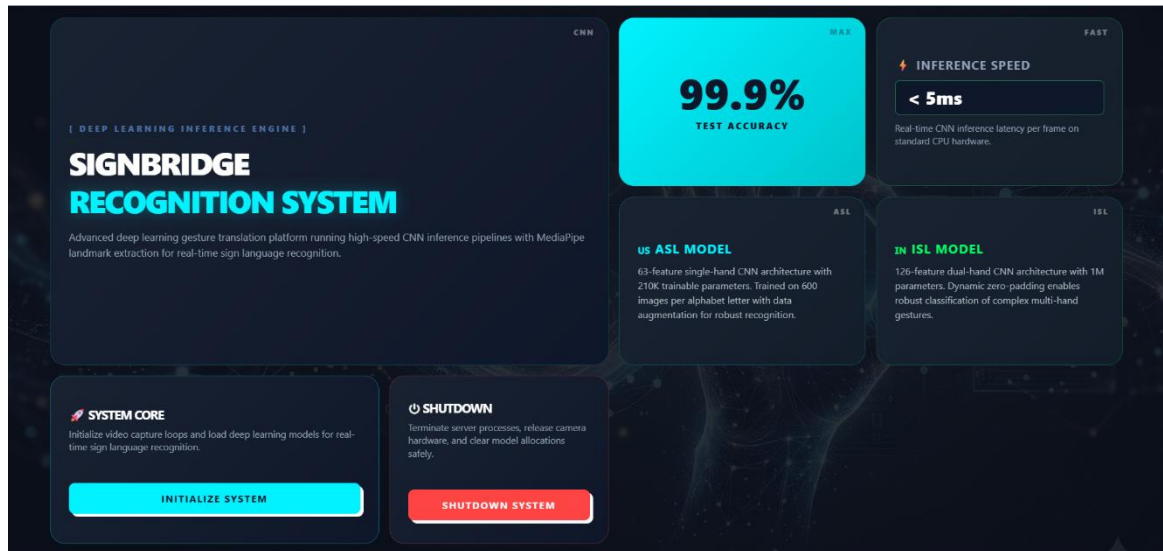


Fig 8.1 Home page



Fig 8.2 Model Selection Page

8.2 OUTPUT SCREEN DESIGN

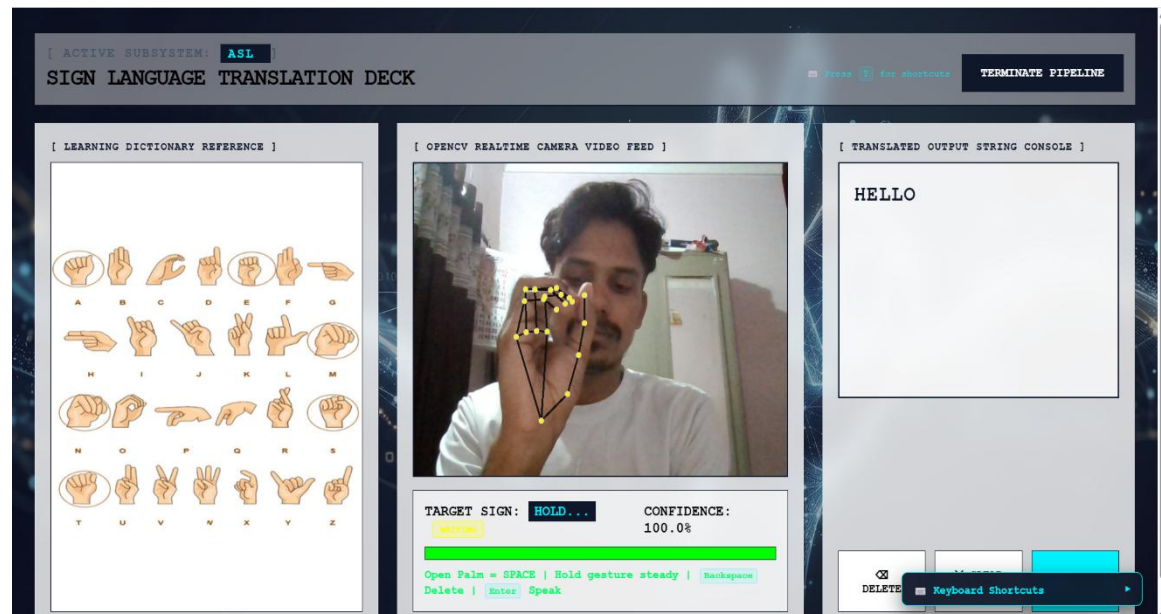


Fig 8.3 ASL Recognition

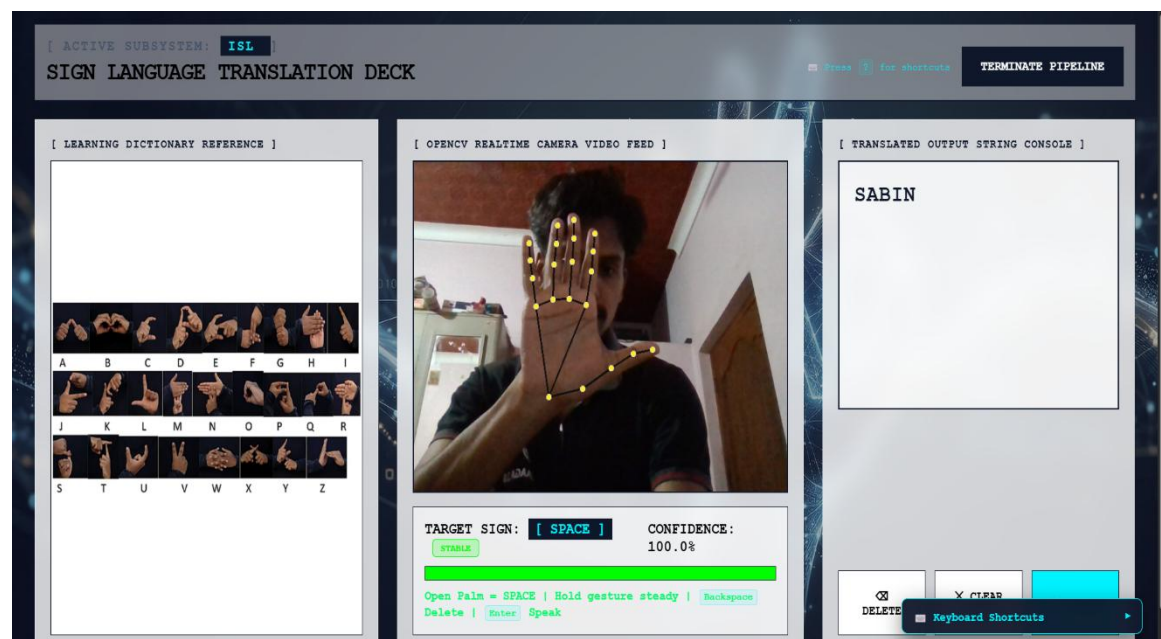


Fig 8.4 ISL Recognition

8.2 SAMPLE CODE

Core Recognition (model_manager.py)

```
import mediapipe as mp

import numpy as np

import tensorflow as tf

from tensorflow import keras

from collections import deque

class SignModelManager:

    def __init__(self):

        self.model = None

        self.mp_hands = mp.solutions.hands

        self.hands = None

        self.labels = {i: chr(65 + i) for i in range(26)}

        self.prediction_history = deque(maxlen=20)

        self.min_confidence = 0.90

        self.consensus_required = 0.75

    def initialize_pipeline(self, mode):

        self.shutdown_pipeline()

        if mode == 'ASL':

            self.model = keras.models.load_model('./saved_models/asl_classifier.keras')

            self.hands = self.mp_hands.Hands(max_num_hands=1)

        elif mode == 'ISL':

            self.model = keras.models.load_model('./saved_models/isl_classifier.keras')
```

```
        self.hands = self.mp_hands.Hands(max_num_hands=2)

    self.active_mode = mode

    return True

def extract_landmark_features(self, hand_landmarks):

    features = []

    wrist = hand_landmarks.landmark[0]

    for lm in hand_landmarks.landmark:

        features.append(lm.x - wrist.x)

        features.append(lm.y - wrist.y)

        features.append(lm.z - wrist.z)

    return np.array(features, dtype=np.float32)

def is_open_palm(self, hand_landmarks, hand_label):

    lm = hand_landmarks.landmark

    thumb_open = lm[4].x < lm[3].x if hand_label == "Right" else lm[4].x > lm[3].x

    fingers = [thumb_open, lm[8].y < lm[6].y, lm[12].y < lm[10].y,

               lm[16].y < lm[14].y, lm[20].y < lm[18].y]

    if sum(fingers) != 5: return False

    dx, dy = abs(lm[9].x - lm[0].x), abs(lm[9].y - lm[0].y)

    return dy > dx * 1.5

def _get_smart_prediction(self, raw_letter, raw_confidence):

    self.prediction_history.append(raw_letter)

    if len(self.prediction_history) < 12:
```

```
        return "Hold steady...", 0.0

    recent = list(self.prediction_history)[-15:]

    consensus = recent.count(raw_letter) / 15

    if consensus < self.consensus_required:

        return "Hold steady...", 0.0

    return raw_letter, raw_confidence

def shutdown_pipeline(self):

    if self.hands: self.hands.close()

    self.hands = None

    self.model = None

    tf.keras.backend.clear_session()

Flask Server (app.py)

from flask import Flask, render_template, Response, jsonify, request

import cv2

from model_manager import SignModelManager

app = Flask(__name__)

manager = SignModelManager()

latest_letter = "Scanning..."

latest_confidence = 0.0

@app.route('/')

def home():

    return render_template('index.html')

@app.route('/recognition')
```

```
def recognition():

    mode = request.args.get('mode', 'ASL')

    manager.initialize_pipeline(mode)

    return render_template('recognition.html', mode=mode)

@app.route('/video_feed')

def video_feed():

    return Response(generate_stream(),

                    mimetype='multipart/x-mixed-replace; boundary=frame')

@app.route('/api/predict')

def api_predict():

    return jsonify({'letter': latest_letter, 'confidence': float(latest_confidence)})

@app.route('/api/terminate', methods=['POST'])

def api_terminate():

    manager.shutdown_pipeline()

    return jsonify({'status': 'Terminated'})

if __name__ == '__main__':

    app.run(host='0.0.0.0', port=5000, debug=False)

CNN Model Architecture (train_complete.py)

def create_asl_model(input_shape=(63,), num_classes=26):

    model = keras.Sequential([

        layers.Input(shape=input_shape),

        layers.Reshape((21, 3, 1)),

        layers.Conv2D(32, 3, activation='relu', padding='same'),
```

```
        layers.BatchNormalization(),  
        layers.Conv2D(32, 3, activation='relu', padding='same'),  
        layers.BatchNormalization(),  
        layers.MaxPooling2D((2, 1)), layers.Dropout(0.3),  
        layers.Conv2D(64, 3, activation='relu', padding='same'),  
        layers.BatchNormalization(),  
        layers.Conv2D(64, 3, activation='relu', padding='same'),  
        layers.BatchNormalization(),  
        layers.MaxPooling2D((2, 1)), layers.Dropout(0.3),  
        layers.Conv2D(128, 3, activation='relu', padding='same'),  
        layers.BatchNormalization(),  
        layers.GlobalAveragePooling2D(), layers.Dropout(0.4),  
        layers.Dense(256, activation='relu'), layers.BatchNormalization(),  
        layers.Dropout(0.5),  
        layers.Dense(128, activation='relu'), layers.BatchNormalization(),  
        layers.Dropout(0.5),  
        layers.Dense(num_classes, activation='softmax')  
    )  
  
    return model
```